



LAWRENCE
LIVERMORE
NATIONAL
LABORATORY

LLNL-TR-813736

Discrete Element Method Analysis for Metal Powders Used in Additive Manufacturing, and DEM Simulation Tutorial Using LIGGGHTS-PUBLIC.

S. Shenouda, A. Hoff

August 19, 2020

Disclaimer

This document was prepared as an account of work sponsored by an agency of the United States government. Neither the United States government nor Lawrence Livermore National Security, LLC, nor any of their employees makes any warranty, expressed or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States government or Lawrence Livermore National Security, LLC. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States government or Lawrence Livermore National Security, LLC, and shall not be used for advertising or product endorsement purposes.

This work performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under Contract DE-AC52-07NA27344.

Discrete Element Method Analysis for Metal Powders Used in Additive Manufacturing, and DEM Simulation Tutorial Using LIGGGHTS-PUBLIC

Unclassified

July 27, 2020

LLNL-TR-813736



Disclaimer

This document was prepared as an account of work sponsored by an agency of the United States government. Neither the United States government nor Lawrence Livermore National Security, LLC, nor any of their employees makes any warranty, expressed or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States government or Lawrence Livermore National Security, LLC. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States government or Lawrence Livermore National Security, LLC, and shall not be used for advertising or product endorsement purposes.

Lawrence Livermore National Laboratory is operated by Lawrence Livermore National Security, LLC, for the U.S. Department of Energy, National Nuclear Security Administration under Contract DE-AC52-07NA27344.

Abstract

Discrete Element Method (DEM) is a method of analysis to evaluate the dynamic interactions between granular particles. This method has been used in the pharmaceutical industry to improve the powder compaction process for tablet manufacturing. There are also applications in agriculture, food industry, and manufacturing. Direct energy deposition is an additive manufacturing technique which uses metallic powders fed through a nozzle, melted using a directed laser, and transformed into a solid object layer by layer. One way of feeding metal particles into the system involves the use of a vibrating hopper. Given a specified amplitude and frequency input, the hopper will enable the powder to travel up a path, and inject through the system with assistance from a stream of gas. The mechanical properties of a printed object can vary, depending on the characteristics of the powder flow and the particles' as-received properties. Improved understanding of dynamic interactions of flowing powders could enable additive manufacturing components with 2D or 3D variations in mechanical properties, e.g., density. This work uses DEM simulation software to investigate the effects of particle cohesion, friction, and density on the quality of the flow by performing an angle of repose simulation, which is often used as a metric to evaluate the flowability of powders.

POWERPOINT SLIDES

Discrete Element Method Analysis

For Metal Powders Used in Additive Manufacturing

Safwat Shenouda – North Carolina A&T State University
Mentor: Andrew Hoff – Lawrence Livermore National Laboratory

June 24, 2020



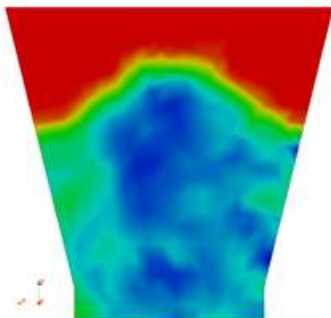
LLNL-PRES-XXXXXX

This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under contract DE-AC52-07NA27344. Lawrence Livermore National Security, LLC.

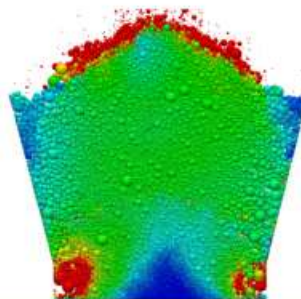
Lawrence Livermore
National Laboratory

Background

- FEM (Finite Element Method)
 - Utilizes a continuum model

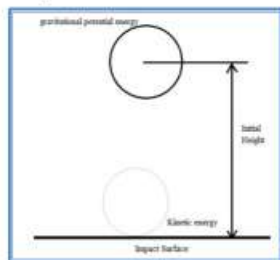


- DEM (Discrete Element Method)
 - Utilizes a discretized model



DEM Fundamentals

- Granular Dynamics: Uses particle kinematics to obtain the incremental displacements from contact reactions



Characteristic Velocity is given by:

$$U_p = mgh$$

$$U_k = \frac{1}{2} m v_{\text{impact}}^2$$

$$mgh = \frac{1}{2} m v_{\text{impact}}^2$$

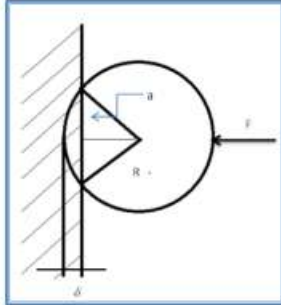
$$v_{\text{impact}} = \sqrt{2gh}$$

- Contact Mechanics: Utilizes theoretical methods to describe force-displacement behavior

Cid, L. D. (2018, August 1). DEM Particle Characteristic Velocity. Retrieved June 3, 2020, from <https://lizardcidphd.com/2017/11/02/dem-particle-characteristic-velocity/>

DEM Contact Models

- **Hertzian Contact Model:** Calculates the contact area and pressure between two surfaces as two objects collide



Using Pythagorean theorem to derive the contact area:

$$\begin{aligned} a^2 + (R - \delta)^2 &= R^2 \\ a^2 &= R^2 - (R - \delta)^2 \\ a^2 &= R^2 - (R^2 - 2R\delta + \delta^2) \\ a^2 &= 2R\delta - \delta^2 \end{aligned}$$

Since δ^2 is very small, it can be omitted, yielding:

$$a^2 = 2R\delta$$

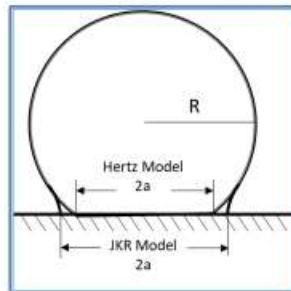
Then the area is defined by:

$$A_0 = \pi a^2 = 2\pi R\delta$$

Cid, L. D. (2018, August 1). The Hertzian Contact Stress Theory. (2018, August 1). Retrieved June 3, 2020, from <https://lizidelcidphd.com/2016/09/12/the-hertzian-contact-stress-theory/>

DEM Contact Models

- **Johnson-Kendall-Roberts Contact Model:** Takes into account the surface energy at contact



Contact radius can be modeled by:

$$a^3 = \frac{R}{E} \left[f_n + 3\gamma_{sur}\pi R + \sqrt{6\gamma_{sur}\pi R f_n + (3\gamma_{sur}\pi R)^2} \right]$$

And maximum tensile force of separation:

$$f_{JKR} = -\frac{3}{2}\pi\gamma_{sur}R$$

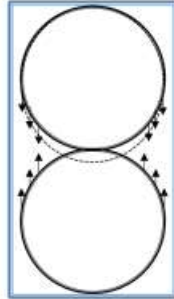
And the total normal force of contact with cohesion:

$$f_n = \frac{4a^3E}{3R} - \sqrt{8\pi E\gamma_{sur}a^3}$$

Cid, L. D. (2018, August 1). The Johnson-Kendall-Roberts Model for Cohesion. Retrieved June 3, 2020, from <https://lizidelcidphd.com/2016/11/21/the-johnson-kendall-roberts-model-for-cohesion/>

DEM Contact Models

- **Derjaguin-Muller-Toporov Model:** An acceptable approximation of cohesion for small particles with weak cohesion



The attractive forces are given by:

$$f_{DMT} = -2\pi\gamma_{sur}R$$

This model uses the Hertzian model, neglecting the contact deflection

Cid, L. D. (2018, August 1). The Derjaguin-Muller-Toporov Model. Retrieved June 3, 2020, from <https://lizidelcidphd.com/2017/07/06/the-derjaguin-muller-toporov-model/>

Methodology

- With LIGGGHTS, we generate an input script to simulate a steel ball in freefall



Notepad or notepad++ can be used to generate input script for preprocessing



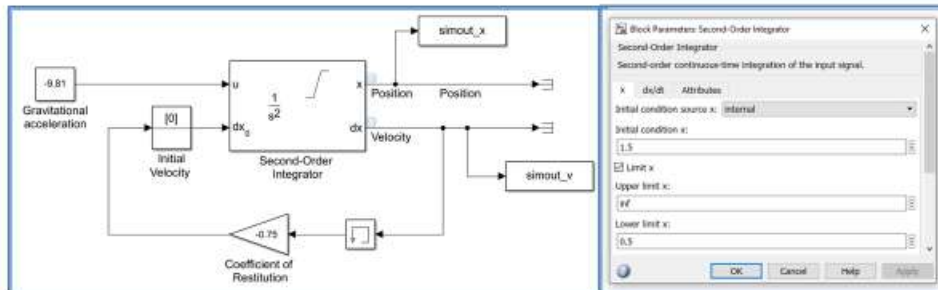
LIGGGHTS uses Linux operating system to run simulation



Blender or ParaView can be used for post-processing/visualization of results

MATLAB Simulation

- Using Simulink, a simple freefall simulation was performed to evaluate the behavior of the LIGGGHTS model



This model uses a second order differential equation, solved using ode45, and will serve as our baseline to compare with the LIGGGHTS simulation

LIGGGHTS Input Script

- Input file (.txt) for LIGGGHTS simulation

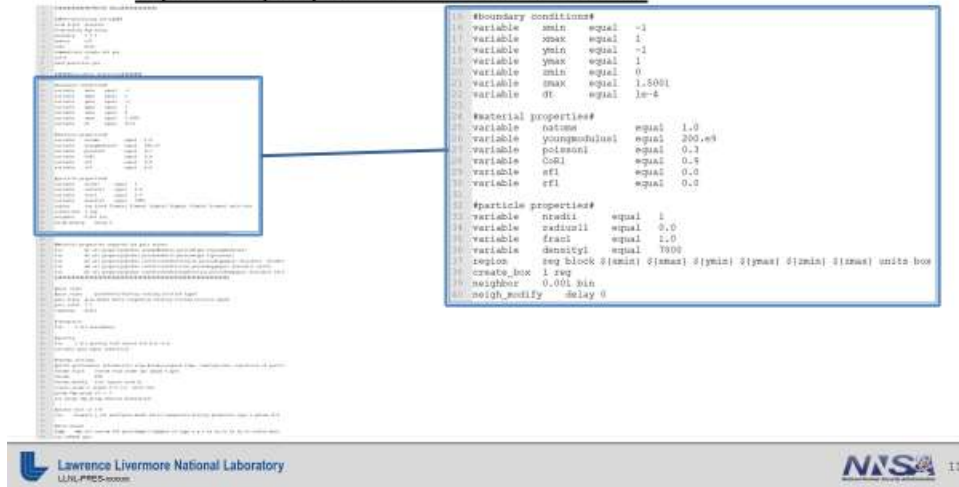
```

1 #####Pre-processing set-up###
2 atom_style granular
3 atom_modify map array
4 boundary f f f
5 newton off
6 echo both
7 communicate single vel yes
8 units si
9 hard_particles yes
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

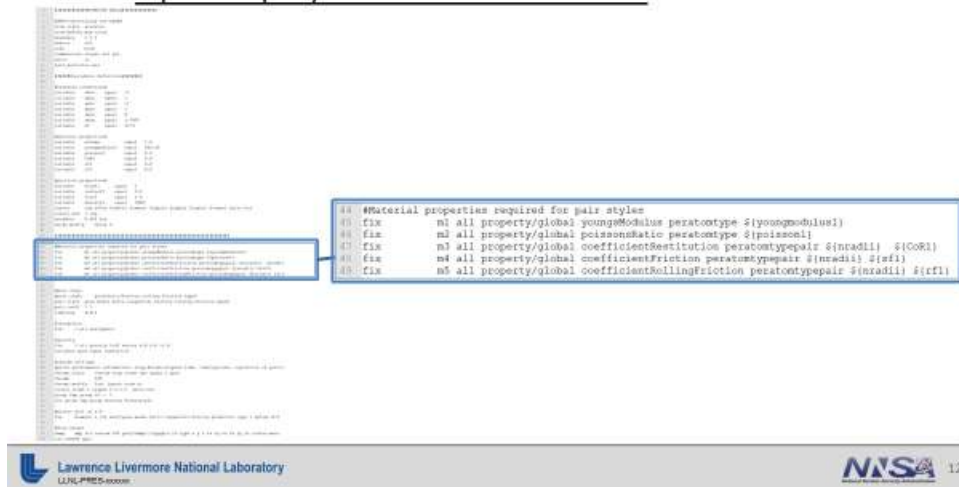
LIGGGHTS Input Script

Input file (.txt) for LIGGGHTS simulation



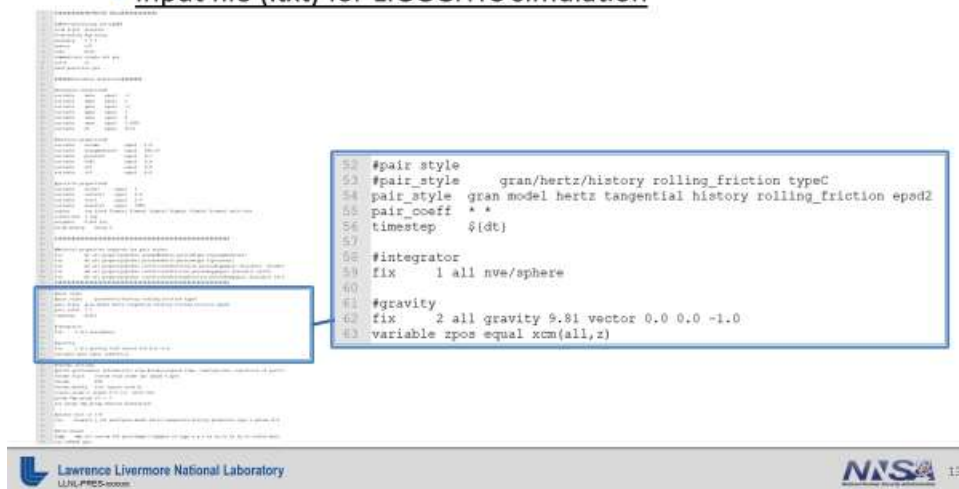
LIGGGHTS Input Script

Input file (.txt) for LIGGGHTS simulation



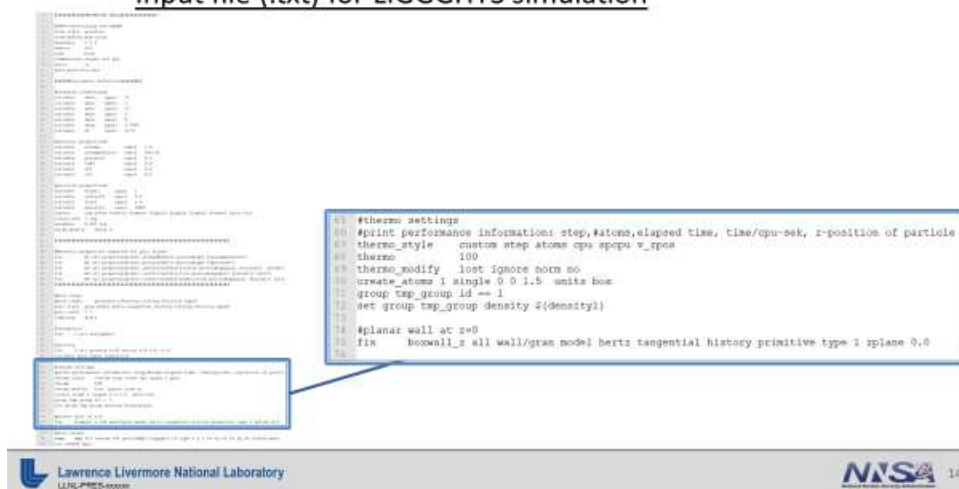
LIGGGHTS Input Script

Input file (.txt) for LIGGGHTS simulation



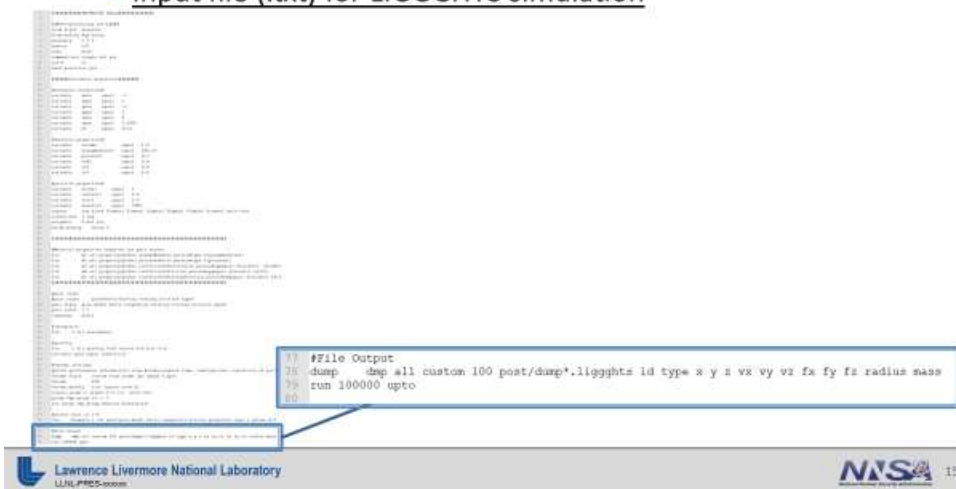
LIGGGHTS Input Script

Input file (.txt) for LIGGGHTS simulation

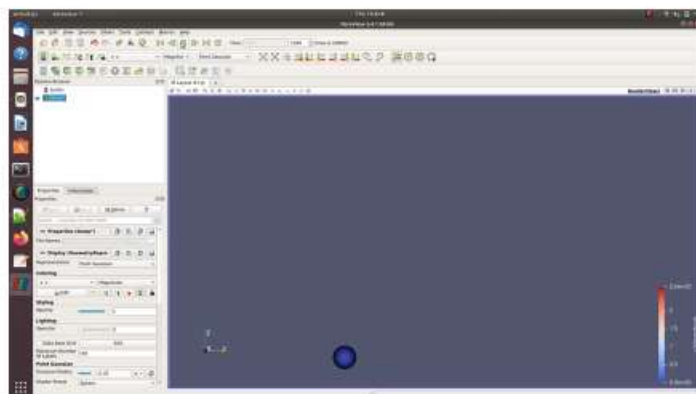


LIGGGHTS Input Script

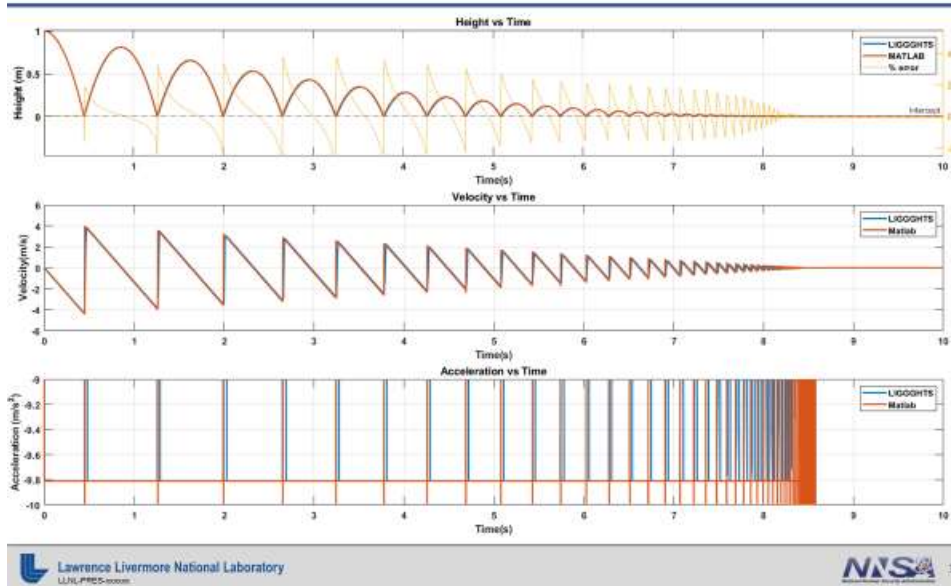
Input file (.txt) for LIGGGHTS simulation



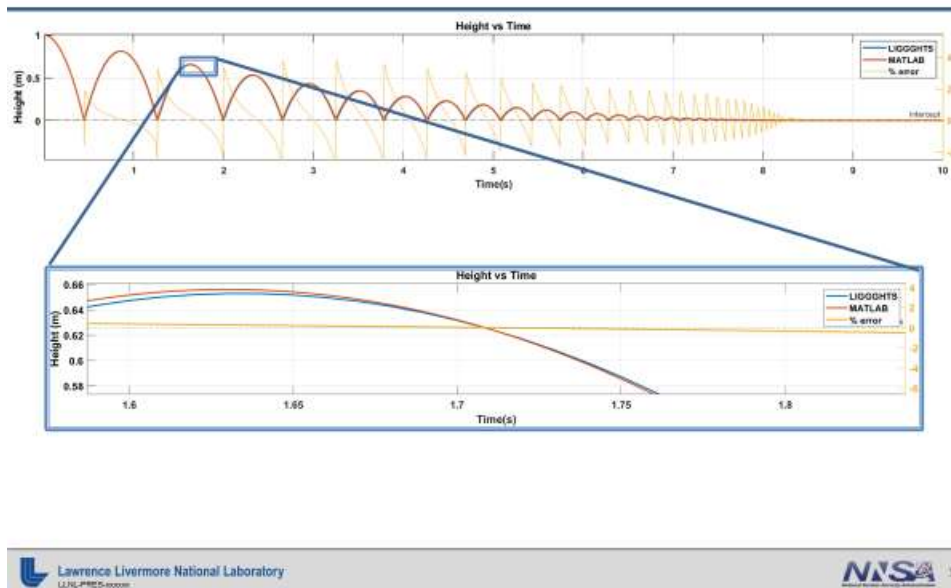
Post-Processing



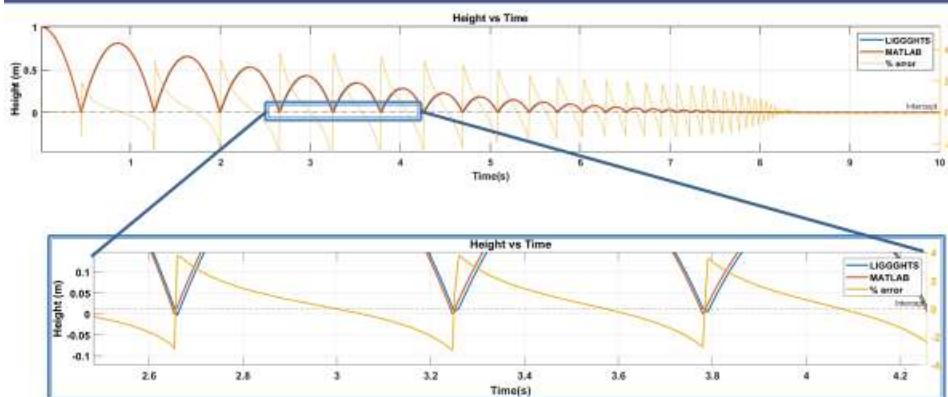
Raw Data Analysis



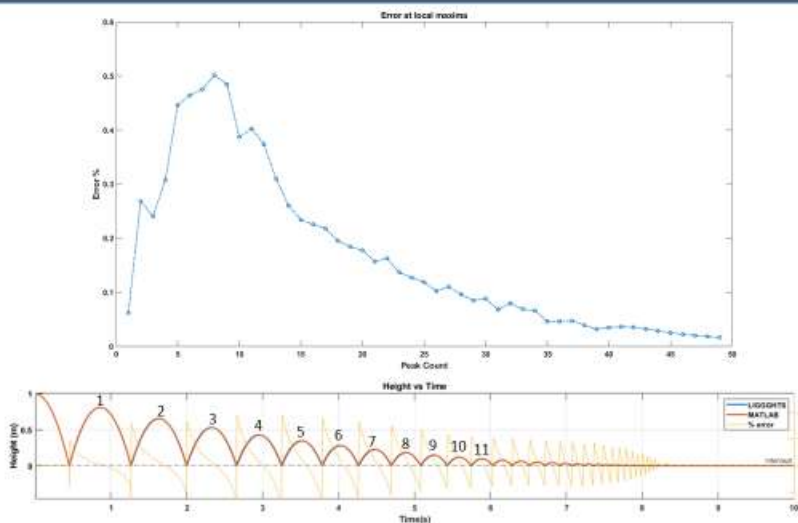
Error Analysis – Local Maxima



Error Analysis – Local Minima



Error Analysis – Local Maxima

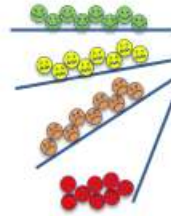


Simulation of Vibratory Feeder

- Analyze Feeding Process of metallic grains in a vibratory feeder system

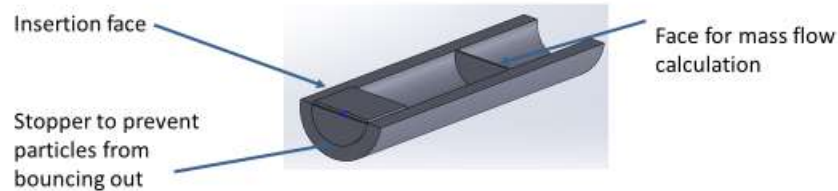


- Simulate how feed rate is affected by ramp incline, input frequency, and input amplitude

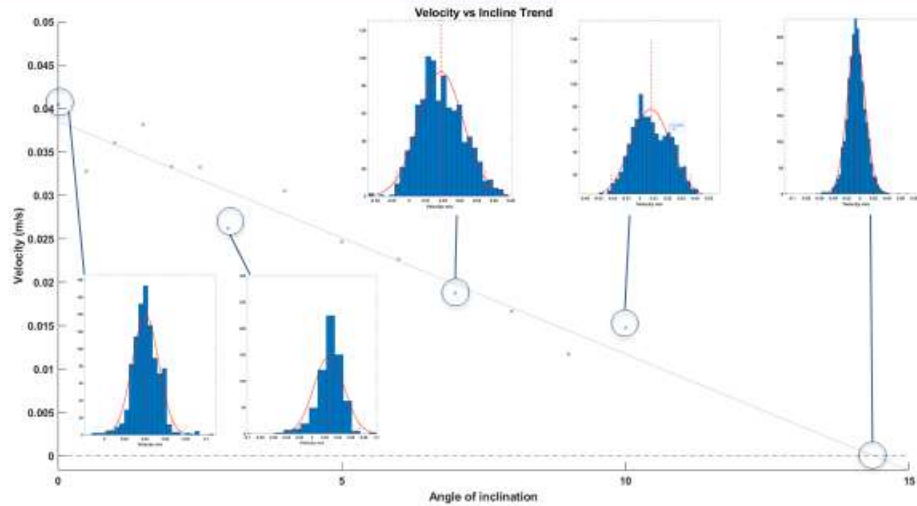


Vibratory Bowl Feeder

- Vibratory feeder simulation
 - Cutting out a small slice of a vibratory feeder due to processing limitations to analyze the flow rate of metallic powder
 - Input frequency of 110-120 Hz, stroke/amplitude TBD



Velocity Trend/Distributions



Future Work

- Analyze trend with changes in frequency
- " " with changes in amplitude
- Compare with data from previous experiments

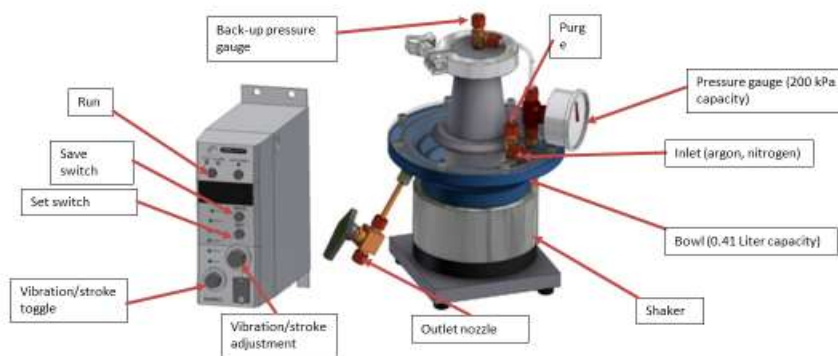
Thank you for your time! Any questions?



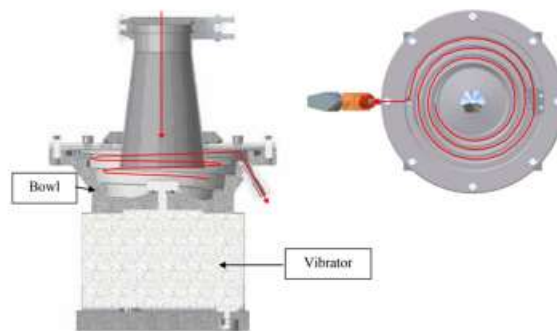
Disclaimer
This document was prepared as an account of work sponsored by an agency of the United States government. Neither the United States government nor Lawrence Livermore National Security, LLC, nor any of their employees makes any warranty, expressed or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States government or Lawrence Livermore National Security, LLC. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States government or Lawrence Livermore National Security, LLC, and shall not be used for advertising or product endorsement purposes.

Feeder System

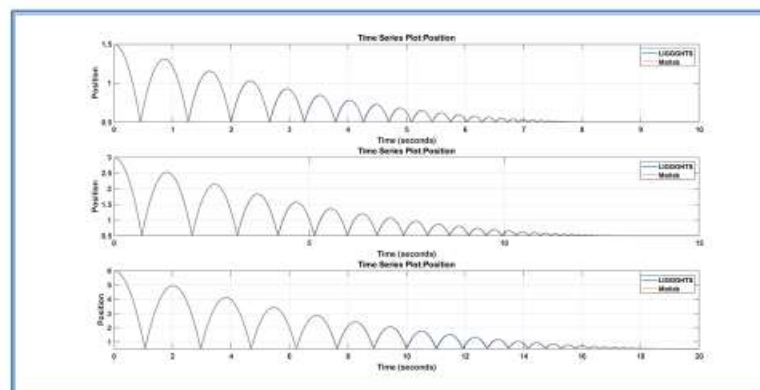
- Vibratory feeder – PFV100-VM-NO by TEKNA Plasma Systems



Feeder System



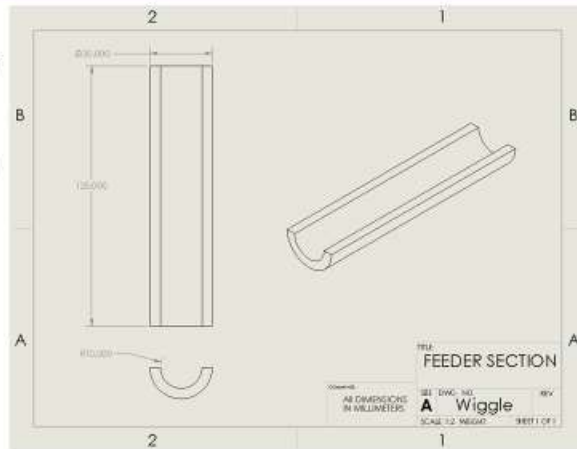
Various Drop Heights



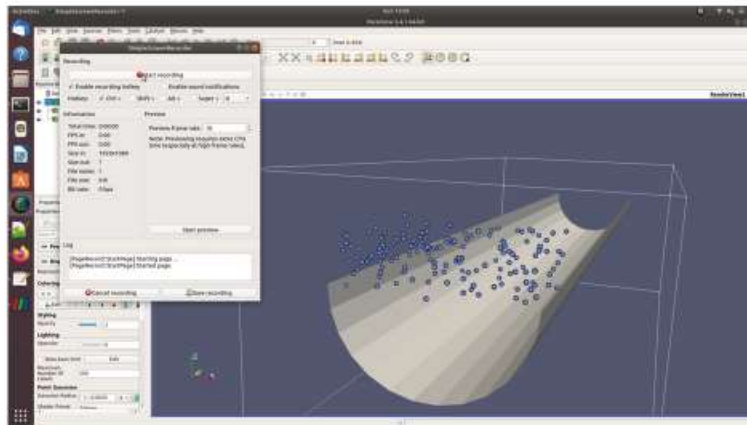
Bounce height comparison at 1.5, 3.0, and 6.0 meters

Vibratory Bowl Feeder

- Vibratory feeder simulation
 - Modeled in Solidworks2019

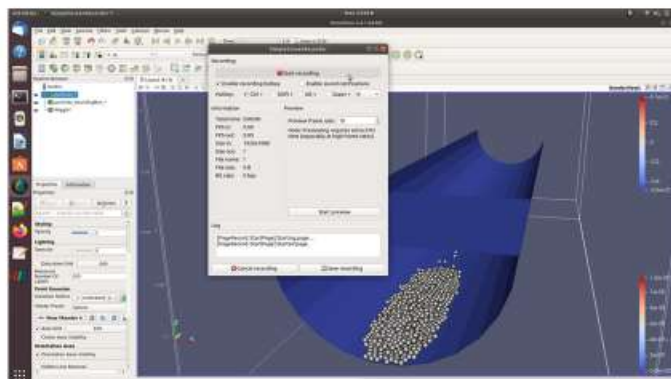


Troubleshooting - No Vibration

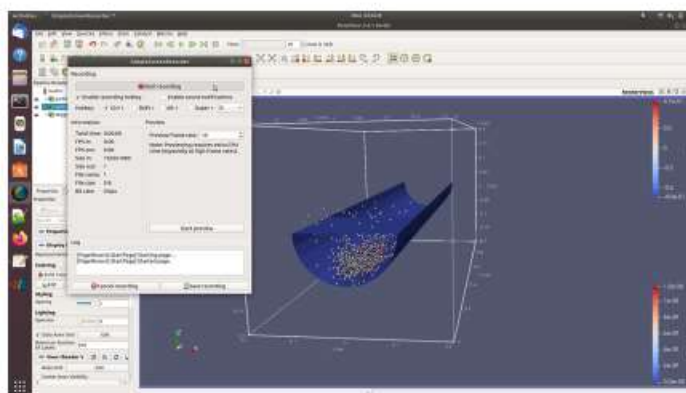


Note the poor adhesion & contact conditions

No Vibration

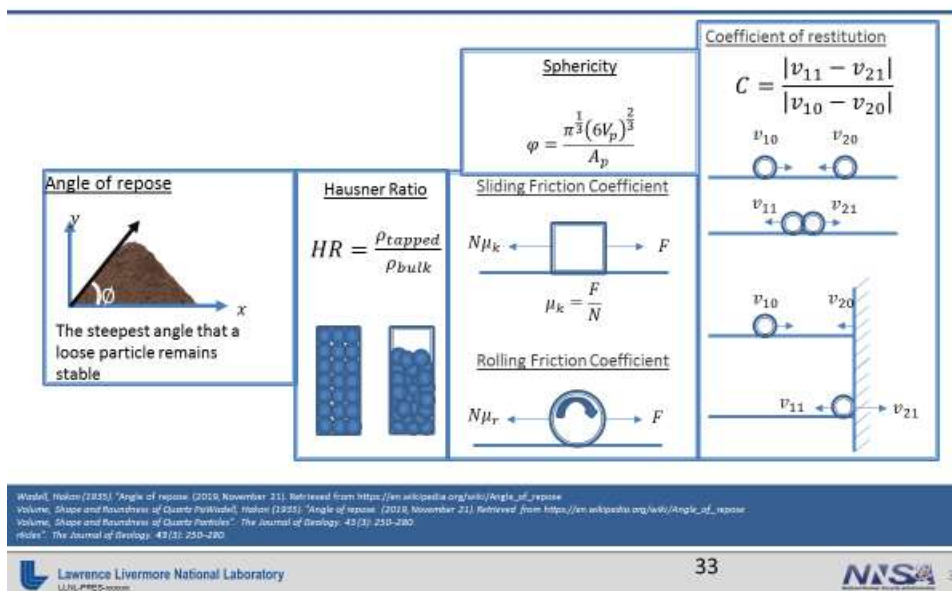


Troubleshooting – With Vibration



Note the poor stability due to high amplitude

Relevant DEM Parameters





Discrete Element Method Simulation Tutorial Using LIGGGHTS-PUBLIC

Unclassified

July 27, 2020



Table of Contents

Section 1. DEM/LIGGGHTS Introduction.....	22
1.1 Overview	22
1.2 Using LIGGGHTS	22
1.3 Post-Processing Using Paraview.....	22
Section 2	23
Section 2. LIGGGHTS Tutorial.....	23
2.1 LIGGGHTS	23
2.1.1 Workflow	23
2.1.2 Commands	23
Summary	24
2.2 The Rayleigh Timestep.....	24
2.2.1 Rayleigh Timestep	24
2.2.2 Hertz Timestep.....	25
2.2.3 Checking for Timestep.....	25
2.3 Particle Creation and Insertion.....	25
2.3.1 Particle Distributions.....	25
2.3.2 Particle Insertion.....	26
2.4 Contact Models.....	28
2.4.1 Hertz	28
2.4.2 Hooke.....	29
2.4.1 Simplified Johnson Kendall Roberts	29
2.4.2 Rolling Friction.....	29
2.5 Movement	30
2.6 Dump Command	30
2.7 Run Command	30
Section 3. Processing.....	32
3.1 Overview	32
3.2 Dumping.....	32
3.3 Error Troubleshooting.....	32
Section 4. Post Processing.....	33
4.1 File Structure	33
4.2 Importing files into paraview.....	33
4.3 Analysis methods.....	34
Appropriate units for SI and CGS	36
SI	36
CGS	37
Results	38
Angle of Repose and Flow Rate With Hertz Model Results	38

Section 1

DEM/LIGGGHTS Introduction

1.1 Overview

The discrete element method (DEM) is used to model the behavior of particles and their dynamic interactions with each other. DEM specifically seeks to replicate the dynamic behavior of particles with their surrounding environment as well as the inter-particle interactions that take place when two particles collide. The dynamic behavior of particles follows general laws of kinematics to determine the incremental displacements of particles as a result of a contact interaction. LIGGGHTS-PUBLIC, or LAMMPS Improved General Granular and Granular Heat Transfer Simulator is a publicly released DEM software for the simulation of granular materials.

1.2 Using LIGGGHTS

Once LIGGGHTS has been installed using the official documentation tutorial, it can be initiated by running a text file through the terminal window. Use the “*cd*” command to navigate to the location of the input script and type in either of the following commands:

```
liggghts -echo both < INPUT_FILE.txt
```

```
mpirun -np 4 lmp_auto < INPUT_FILE.txt
```

The first command is usable for practically all situations. When activating multiple processor cores using the *processors * * ** command, it is necessary to use the *mpirun* method. Prior to using this method, the file titled “*lmp_auto*” must be inside of the same directory as the input script. The *lmp_auto* file can be found inside of the *home/LIGGGHTS-PUBLIC/src* directory, and can simply be copy/pasted into the input script’s directory. After the simulation is initialized, there will be a *post* folder inside of the input script’s directory containing a series of dump files with the file types *.vtk*, *.liggghts*, and *.stl* for geometries. There are also example simulation files to run, as well as tutorials on youtube. The “engineerdo” silo tutorial is especially helpful.

1.3 Post-Processing Using Paraview

Paraview 5.4.1 is the recommended version to use as the new version, Paraview 5.8.0, has fewer capabilities. The main functionality difference is that Paraview 5.4.1 is readily compatible with the installable *.liggghts* plugin to view the dump files, whereas 5.8.0 is only compatible with the *.vtk* files. The older release version also has a setting to scale particles down to their true radius sizes. The file compatibility can be remedied by outputting the simulation files using the *.vtk* format, which will be explained later.

Section 2

LIGGGHTS Tutorial

2.1 LIGGGHTS

LIGGGHTS-Public is an open-source linux simulation software that utilizes .txt input files to simulate the behavior of granular materials. Several contact models used include the Hertz model, Hooke model, and the simplified Johnson-Kendall-Roberts theory for particle cohesion.

2.1.1 Workflow

The software searches the text input file at each line for a command and relevant arguments, then moves on to the next one. The simulation gets initialized through a linux terminal command, and raw text output files are populated in a *post* folder to be imported into a post-processing software such as ParaView or Blender.

The basic workflow is:

1. Generation of .txt input file

- a. Define particle type, units, and variables
 - b. Define geometries, and boundary regions
 - c. Define computational parameters, including:
 - i. Neighboring commands
 - ii. Contact models and cohesion
 - iii. Time-step and output parameters
 - d. Particle creation, distribution, and insertion
2. Calibrate model and adjust input parameters
3. Post processing of data into a visual/graphical form
4. Analysis of raw data using Excel, or Matlab, or other computational software

2.1.2 Commands

The syntax of LIGGGHTS follows a basic command structure. First a command is defined, then the command ID is given, followed by any arguments. Commands most frequently used in all simulations are known as general commands. This includes the *fix*, *variable*, *region*, *dump*, and *run*. There are additional commands such as *cohesion*, *compute*, *pair_style*, and *model*. These command types may rely on the definition of additional physical properties, and are used for specific situations depending on which model type is being used. Below is an example of defining material properties using variables that can be adjusted to reiterate a simulation for model calibration.

variable modulus1 equal 1e7 #defining a variable
general command variable string argument value comment

User designated value and group ID

fix m1 all property/global youngsModulus peratomtype \${modulus1} #youngsmodulus (MPa)
general command specific command command parameter argument variable string comment

The variable previously defined can be recalled using $\${string}$. It is also possible to use equations by inserting basic operators such as $\${string1}/\${string2}$, or $\cos(\{string1\})$. The benefit of defining variables to be recalled later is that it makes it easier to adjust parameters of a simulation between multiple iterations by simply changing the variables.

Summary

More information can be found regarding commands and their use on the cfdem online documentation page at <https://www.cfdem.com/media/DEM/docu/Manual.html>. The website gives command types, their structures, and their use. It is a good practice to open an example script, and look up each command in the documentation to determine the proper syntax and usage of each command.

2.2 The Rayleigh Timestep

The timestep chosen when performing a simulation is of major importance in all DEM simulations, and determines not only the stability of a simulation, but the amount of time it takes to perform a simulation. An equation can be formed to determine the amount of steps to run a simulation in the input script. To determine the steps taken use the following:

$$\frac{\text{Simulation period (s)}}{\text{timestep}} = \text{Simulation steps}$$

A standard practice is to start at the Rayleigh timestep and calibrate the timestep for the simulation until the performance is optimal. Usually, the timestep is selected to be around 20% percent of the Rayleigh timestep. The hertz timestep may be used when the hooke model is being used as a pair style.

2.2.1 Rayleigh Timestep

The Rayleigh timestep for a DEM simulation is calculated using the following equations:

$$G = \frac{E}{2(1 + \nu)}$$

$$\frac{1}{R} = \frac{1}{R_1} + \frac{1}{R_2} + \frac{1}{R_n} \dots \quad \frac{1}{E} = \frac{1}{E_1} + \frac{1}{E_2} + \frac{1}{E_n} \dots$$

$$\text{Rayleigh Timestep (dt)} = \frac{\pi R \sqrt{\frac{\rho}{G}}}{0.1631\nu + 0.8766}$$

Where E = Young's modulus, G = shear modulus, ρ = density, R = particle radius, and ν = poisson's ratio.

2.2.2 Hertz Timestep

The hertz timestep may be also be used, yet determining the value may be too difficult as maximum particle velocity needs to be known prior to its use. The equation for the hertz timestep is:

$$\text{Hertz Timestep } (dt) = 2.87 * \left(\frac{m^2}{RV_{max}E^2} \right)^{0.2}$$

Where m = mass and V_{max} = maximum velocity.

2.2.3 Checking for Timestep

As mentioned previously, the timestep is highly important in order to reduce the computational load and increase the stability of a simulation. Simply calculate the Rayleigh timestep, run a simulation and calibrate the value accordingly based on the performance. A basic command line can be added to the input script which calculates the timestep every n steps and outputs the current Rayleigh timestep percent in the terminal window.

```
fix ts_check all check/timestep/gran 1000 0.2 0.2 #check timestep
```

This command will check the timestep every 1000 steps, and output a warning if the value is greater than 20% of the Rayleigh time and Hertz time, respectively.

2.3 Particle Creation and Insertion

Much of the time calibrating a DEM simulation will be spent optimizing the amount of particles, their size, and the rate of insertion. There is generally no particle limit but it is mostly dependant on the available computation power, the size of the domain, and the size of the insertion region. The terminal window will prompt a warning when the *thermo* command is detected within the script, and will output various parameters including the overall kinetic energy of the system, the amount of particles inserted, and the elapsed simulation steps. The simulation will complete when the steps have reach the desired step, defined by the *run* command. If the kinetic energy is varying wildly, it is a sign of an unstable simulation. To fix this, either insert less particles, insert the particles more gradually, or alter some of the mechanical properties. The amount of particles can be inserted in two ways, either by mass or by the amount of particles. In addition, there are three different insertion methods which can be used.

2.3.1 Particle Distributions

When creating a particle distribution, the amount of different radius sizes needs to be determined. To initialize the distribution use the following commands:

```
variable nradii equal 3
variable radius1 equal 0.0025
variable radius2 equal 0.0035
variable radius3 equal 0.005
variable density equal 2.6
variable frac1 equal 0.5
variable frac2 equal 0.25
variable frac3 equal 0.25
```

This will create a distribution of 50% of particles with a radius of 0.0025, 25% particles with a radius of 0.0035, and 25% particles with a radius of 0.005. To initialize the particle creation, a *fix* is created of the type *particletemplate/sphere*, as shown below.

```
fix pts1 all particletemplate/sphere 10487 atom_type 1 density constant ${density} radius constant
${radius1}
fix pts2 all particletemplate/sphere 11887 atom_type 1 density constant ${density} radius constant
${radius2}
fix pts3 all particletemplate/sphere 11897 atom_type 1 density constant ${density} radius constant
${radius3}
fix pdd1 all particledistribution/discrete 32452867 ${nradii} pts1 ${frac1} pts2 ${frac2} pts3 ${frac3}
```

An important thing to note in this command are the numbers 10487, 11887, and 11897. These are random numbers used to generate the particles. LIGGGHTS uses a monte carlo method to randomize the particle insertion until the volume has been filled. The requirement for these random numbers is that they are unique, prime numbers, and greater than 10000. A prime number can easily be generated in MATLAB using the *primes(n)* command, where *n* is the maximum value. For example, *primes(20000)* will generate all the prime numbers until 20000. Altering these numbers between simulations will generate statistically similar values, similar to running multiple experiments with varying results. If an incorrect value is chosen, LIGGGHTS will output an error in the terminal window, and provide an example of numbers that can be used. They can be copy/pasted into the input script.

2.3.2 Particle Insertion

There are three methods for inserting particles into a simulation domain. Two of the most effective methods are the *insert/stream* method and the *insert/rate/region* methods. Particle insertion using the *stream* method required an .stl file generated from solidworks or some other CAD program, whereas the *region* method required a previously defined geometry. The *region* method is the most convenient since the insertion region defined by the geometry's dimensions are easily edited. The *insert/pack* is used to fill a volume with particles up to a given volume fraction. This is the least efficient insertion method so is only used when the need arises.

2.3.2.1 Stream Method

The *insert/stream* method for particle insertion depends on the generation of an .stl file from a CAD program. Particles will generate at the region of the stl file and use it as an opening to be inserted. Similar to the particle distribution, a random prime number greater than 10000 must be used to generate the seed values.

```
fix ins all insert/stream seed 86028157 distributiontemplate pdd1 mass ${fillmass} massrate${fillmassrate}
overlapcheck yes all_in yes vel constant 0 0 -0.75 insertion_face ins_mesh extrude_length 0.1

fix ins all insert/stream seed 86028157 distributiontemplate pdd1 nparticles ${n} particlerate ${n}
overlapcheck yes all_in yes vel constant 0 0 -0.75 insertion_face ins_mesh extrude_length 0.1
```

The amount of particles inserted can either be defined as the mass $\left(\frac{4\pi n \rho R^3}{3}\right)$ and mass rate, or by the amount (*n*) of particles and the particle rate. In addition, it is good practice to use the command *insert_every* *\${timesteps}* after using *massrate/particlerate* to define how often the particles will be inserted, as well as defining the amount of particles to be INF, for infinite. For example, using *nparticles INF particlerate 1000 insert_every 100* will generate 1000 particles every 100 steps until a predetermined amount of timesteps have elapsed. Particle insertion is another source of instability in DEM simulations. If particles are

generated too quickly, they will overlap, bounce away wildly, or prompt an error in the terminal window. In this case, either insert fewer particles, insert them more gradually, or increase the size of the insertion region. It is important that the particles generate and clear the insertion region fast enough to provide space for additional particles to generate. Increasing the initial velocity is performed with the `vel constant  $\dot{x}$   $\dot{y}$   $\dot{z}$` command, where the values are a three dimensional velocity vector. The size of the region can be increased using the `extrude_length  $x.x$` command, where the extrusion is normal to the face of the insertion geometry.

2.3.2.2 Rate/region method

The `insert/rate/region` method for inserting particles uses similar syntax as the `insert/stream` method, but instead of requiring an .stl file, a geometry can be generated within the script using the `region` command. This method makes editing the geometries much simpler, because if something is wrong with the geometry, it can be edited directly within the script.

2.3.2.3 Insertion Faces

Particle insertion requires defining a plane, or region that they will be inserted from. Two methods of doing this is by importing an .stl planar geometry, or generating a 3 dimensional region which the particles will come from. Both methods require that the geometry will fit within the created bounding box, and be able to fit all of the particles with no overlap. If an .stl geometry doesn't fit within the bounding box, using movable boundaries `boundary  $m m m$` or `boundary  $p p p$` , or using the `scale` command to scale the geometry to a smaller size may work. To insert an .stl file use the following `fix` commands:

```
fix    ins_mesh all mesh/surface/planar file Insertionsface.stl type 1 scale 0.001
fix    ins all insert/stream seed 86028157 distributiontemplate pdd1 mass ${fillmass} massrate
${fillmassrate} overlapcheck yes all_in yes vel constant 0 0 -0.75 insertion_face ins_mesh extrude_length
0.25
```

Alternatively, a region can be created and particles can be inserted using the following `fix` commands:

```
region factory cylinder z 0 0 0.0075 0 0.3 units box
```

<https://www.cfdem.com/media/DEM/docu/region.html> contains more details on which types of geometries can be inserted. For this cylinder, a 3-dimensional cylinder will be created along the z axis, and the centroid is along the vertical line that intersects the point $(x,y) = 0,0$. The radius is 0.0075, and the cylinder is bounded by the 2 points along the z axis at 0.0 and 0.3. Figure 1 shows a depiction of the cylindrical region formed by this command.

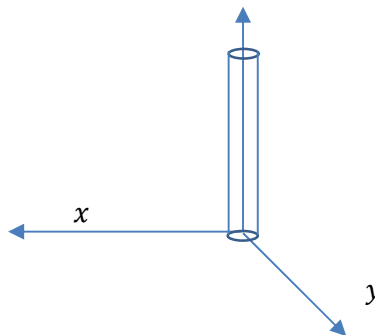


Figure 1: Inserting a cylinder for particle insertion

2.4 Contact Models

The definition of contact models are required for all DEM simulations in LIGGGHTS. These models calculate the incremental displacements as a result of a collision between two or more particles. The contact models function by calculating the surface area between two particles as a function of the overlap distance, normal and tangential forces, and the velocities of the collision. Some contact models require defining additional variables to work, including the Hooke model, which requires an estimated velocity as a specific point of interest. For example, for a group of particles exiting a silo, the velocity at the exit point is used. In addition, there are cohesive models that also need to be defined such as the Simplified Johnson Kendall Roberts model which requires the cohesive energy density be defined as a material property. There are also *hertz/stiffness* and *hooke/stiffness* models available that require normal, tangential modes of stiffness and damping coefficients. Since these values require additional analysis to compute, they will be ignored until they can be determined. More information is available at:

https://www.cfdem.com/media/DEM/docu/Section_gran_models.html?highlight=contact%20model.

It is important to search the whitelist (Figure 2) located inside the *src* directory in the LIGGGHTS-PUBLIC folder. This will tell you any of the possible combinations of models that can be used. If an incorrect combination is used, LIGGGHTS will prompt an *error: Contact model not found in any whitelist*. This will either run unoptimized, and may not be able to simulate the correct dynamics of the system.

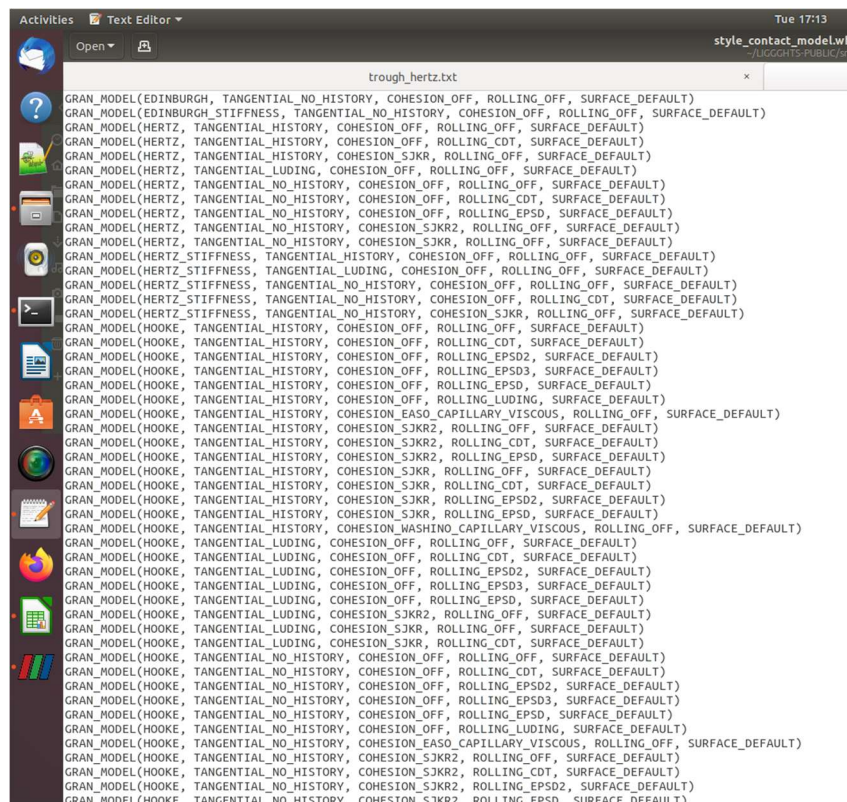


Figure 2: Whitelist for model combinations

2.4.1 Hertz

This model determines the contact area defined by two colliding particles as a function of the overlap between them. The contact area is derived using the Pythagorean theorem to yield the following equation:

$$A_0 = 2\pi R\delta$$

Where δ = the overlap between the two particles. This contact model neglects the deformation at the contact area between two surfaces. The use is as follows:

```
pair_style gran model hertz tangential history rolling_friction epsd2
```

This command will also include the tangential forces that occur at the surface of a particle and store them using the *history* command, which is used to determine the resulting angular velocity after an impact between two particles. The *rolling_friction* command includes the effects of rolling friction, requiring a rolling friction coefficient to be set in the material properties section. When using contact models, it is important to use the same models for the geometry. To add a contact model to the geometry use:

```
fix walls all wall/gran model hertz tangential history rolling_friction epsd2 mesh n_meshes 1 mesh cad1
```

2.4.2 Hooke

Syntax and use of the Hooke model is the same as the Hertz model. However, when using the Hooke model, the characteristic velocity must be determined. The characteristic velocity may be estimated through experimentation, numerical methods, or running a simulation with the Hertz model, and using the velocity for the Hooke model.

2.4.1 Simplified Johnson Kendall Roberts

The simplified Johnson Kendall Roberts (*sjkr*) and its modified version (*sjkr2*) can be used interchangeably. These models include the deformation due to a particle collision, and needs to be preceded by the *cohesion* command.

2.4.2 Rolling Friction

The rolling friction command is simply to allow the software to include the effects of rolling friction in its calculations. Various modifications to this can be performed using the *cdt*, *epsd* *epsd2*, or *epsd3* rolling friction commands. This model requires that the rolling friction be fixed as a material property using the following syntax:

```
fix m1 all property/global rollingFriction peratomtypepair ${nobjects} ${rf11} ${rf12} ${rf21} ${22}
```

The *nobjects* variable represents the amount of different object types within the simulation. The particles and the inserted geometries count each count as one object type, so two would be defined as the variable. The following four numerical properties represent the rolling friction coefficients from particle-to-particle, particle-to-geometry, geometry-to-particle, geometry-to-geometry. Two important details are that the particle-to-geometry and geometry-to-particle coefficients should be equal, geometry-to-geometry can be zero since there are no geometries acting on each other.

2.5 Movement

Moving an object such as a lid, shaking a boundary, or rotating a drum is done with one single command. There are multiple types of movement including rotation, translation, or vibration. They will run up to a predefined step, and may be unfixed to cancel the motion. A fix command is used to define the motion parameters. For example, using the wiggle command to shake an object with a given amplitude vector and frequency:

```
fix move all move/mesh mesh cad1 wiggle amplitude 0. 0. 0.001 0.001 period 0.01
```

This results in a vibration with an amplitude vector of $(x,y,z) = (0,0.001,0.001)$ with a frequency of 100 Hz $\left(\frac{1}{0.01}\right)$. This will shake until the *unfix move* command is performed. Other movement commands could be explored at https://www.cfdem.com/media/DEM/docu/fix_move_mesh.html?highlight=move.

2.6 Dump Command

The dumping process will output a raw text file every n timesteps for each geometry and particle group. These files will be placed in a folder, designated as “post” so they can easily be found. First, the *shell mkdir post* command will create a folder in the input files directory called “post”. The amount of files is dependant on how many objects have been inserted into the simulation. The following command will dump files, and make sure the correct dynamic parameters are calculated:

```
dump dmpparticle all custom/vtk 1000 post/particles_*.vtk id type x y z vx vy vz fx fy fz radius mass
```

This will generate a .vtk file every 1000 steps that represents the particles displacement, velocity, force, radius, and mass. In order to dump .liggghts format files use the following:

```
dump dmpparticle all custom 1000 post/particles_*.liggghts id type x y z vx vy vz fx fy fz radius mass
```

Although there is no functional difference between .vtk and .liggghts files, the benefit of .vtk files is that it will output the bounding box of the simulation into post-processing. All versions of paraview are compatible with reading .vtk files, but to use the .liggghts reader, Paraview 5.1 needs to be used.

2.7 Run Command

The run command initializes the input script processing. LIGGGHTS checks each line at a time while running the parameters through the software. The command for running a simulation is:

```
Run 1150000
```

This will run the simulation up to 1150000 timesteps. There is no fundamental limit for this, but the amount of timesteps is limited by hard drive space. To determine the amount of files that will be dumped for post-processing, divide the steps defined by the run command by the dump steps. This is the amount of files that will be generated for each atom type. Atom types represent the particles, and geometries. If there is 1 particle type and 1 geometry, a run of 1150000 time steps dumping files every 1000 steps will produce $\left(\frac{1150000}{1000} * \right) * 2 = 2300$ files. Static objects inserted such as bounding boxes and insertion faces

will only generate a single file. If an object needs to be removed after a certain amount of time steps, the *unfix object_name* can be inserted. Any command defined by a *fix*, *region*, *variable*, or *move* command will disappear from the simulation.

Section 3

Processing

3.1 Overview

Processing begins the moment the simulation is run. After the *run* command has been entered, LIGGGHTS will output a series of values regarding the current timestep, kinetic energy, and other user-defined output parameters. Information about the simulation parameters will be presented as the software runs through the input file line-by-line every 100 timesteps.

3.2 Dumping

As the simulation runs, the files will be dumped into the post folder, where they can be added to Paraview and viewed. This will help to view the simulation while it simultaneously runs in the background. In order to do this correctly, the contents of the post folder needs to be empty between simulations. After a simulation is run, rename the post folder so it will not be overwritten by a new simulation.

3.3 Error Troubleshooting

LIGGGHTS will perform a check for errors during each line of the input script. To troubleshoot an error, read the command output in the terminal window for information. The last line before the error will be a good place to start checking for errors. Random characters can be placed in the script to induce an error to find out which command induces an error. A full list of errors can be found at https://www.cfdem.com/media/DEM/docu/Section_errors.html.

To help ensure a trouble free simulation, make sure the correct units and values are input into the script. There are fundamental limits to the size of the particles. Make sure the values defined for the parameters are appropriate for the units.

Section 4

Post Processing

4.1 File Structure

After a simulation is performed, files are created for each dumped timestep within the *post* folder. Also *.stl* files and their move steps will be located here. Inside of the inputs directory, a log file is also created. The log file echoes the data from the terminal window as the simulation gets processed. Within the first section of the log file is a copy of the input script for the particular simulation. The second section of the log file is the computational data, including the inserted particles, timesteps, and kinetic energy of the system. The input directory also contains the *.stl* mesh files, run scripts, and any other data needed for a simulation.

4.2 Importing files into paraview

LIGGGHTS will place all the simulation files in the *post* folder and arrange them in order by timestep. When outputting simulation files as *.vtk*, the bounding box will be it's own subset of files, along with the particles and any geometries. When using the *.liggghts* file format, only the particles and geometries will be saved. To add these in, go to *open>input_script_location>post_folder* and add all of the files, then press apply to place them into the layout window.

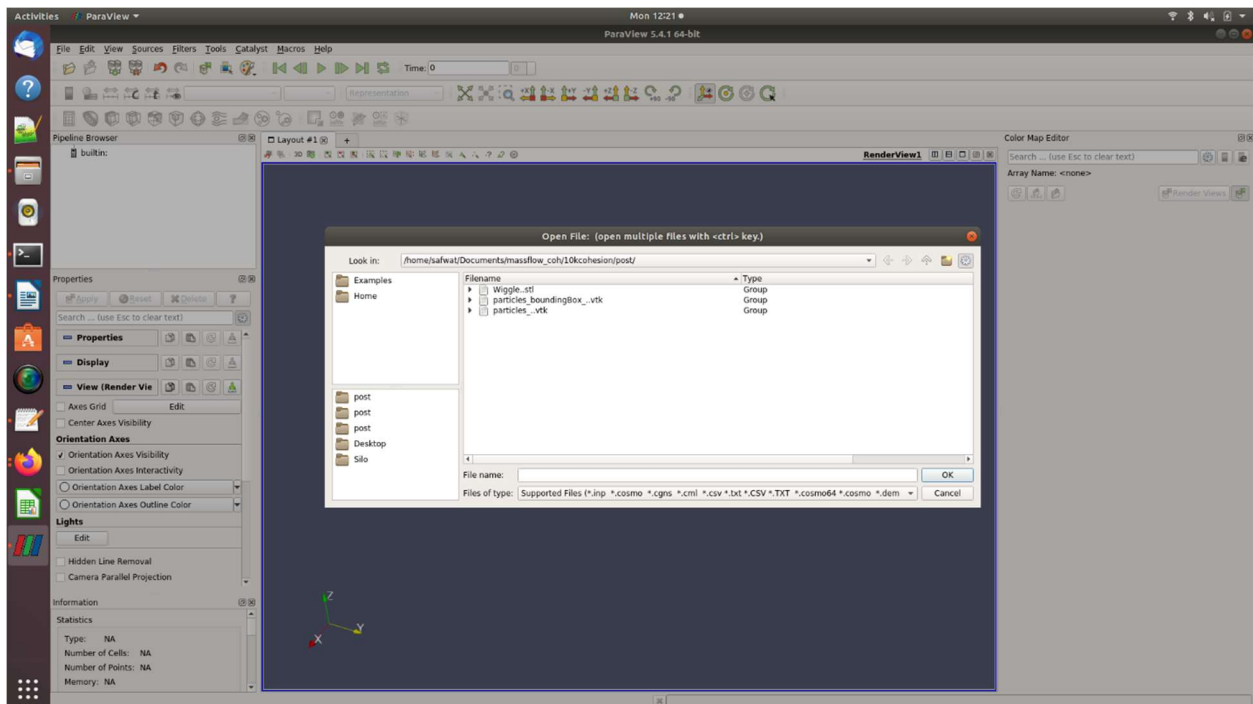


Figure 3: Importing *.vtk* simulation files into ParaView

Initially, the particles will be populated as white point particles. The particles can be changed to be displayed according to their force, velocity, point ID, tangential velocity, radius, or mass. Point Gaussian is the preferred method to view the particles at their actual sizes. Select the particles in the pipeline

browser, and allow the simulation to run for a few steps then pause it. In the property window, select 1 as the Gaussian radius, then click the box “use scale array” and set radius as the Gaussian scale array. This option is only available in ParaView version 5.4.1, which is why it is the ideal version to use.

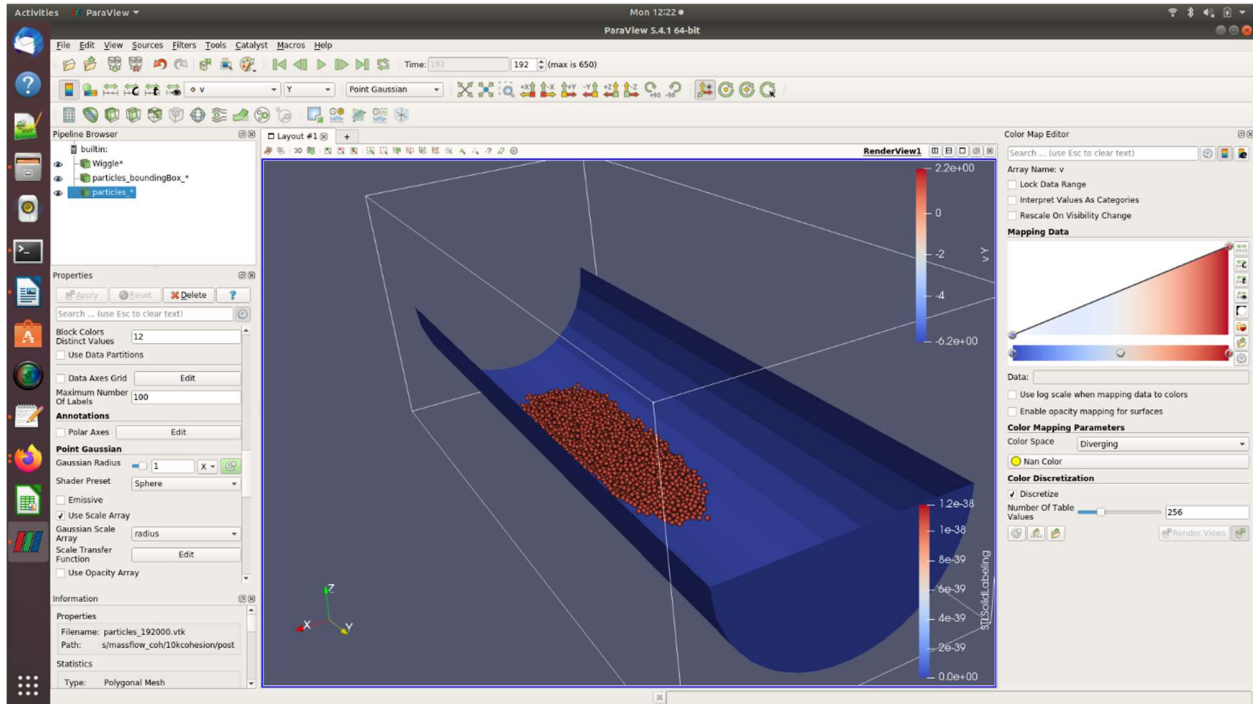


Figure 4: ParaView GUI

4.3 Analysis methods

To begin analyzing the raw data, the file can be saved within ParaView. The “+” next to “Layout #1” can be pressed to open a new layout editor. Choosing the “Plot Matrix View” will display a statistics editor, which may be populated with any desired variables.

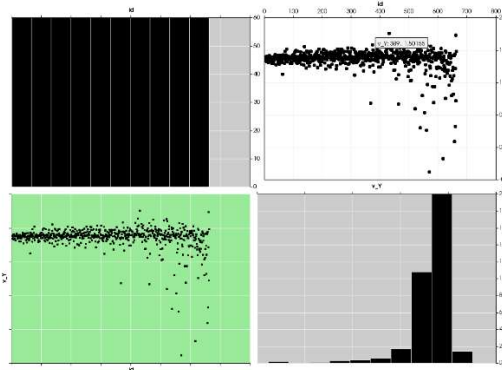


Figure 5: Velocity distributions

Alternatively, the raw data file can be saved as a .csv file, which can be imported into Excel or Matlab for analysis or further post processing. This is performed by *File>Save Data*. In order to evaluate the mass flow rate of particles, an extra set of commands are placed in the input script. This command requires that a planar object be created using a CAD program. It works by counting how many particles pass through the face, their sizes, and their masses.

fix mass all massflow/mesh mesh plane 1 count once delete_atoms no vec_side 0. 0.625 0. file massflow.txt

This will output a raw data .txt file that can be imported into Excel. The first column represents an identifier that represents which particle has crossed the boundary, while the second column represents the diameter of that particle. The 7th column represents the mass flow (grams/second) of the particles crossing the face.

	A	B	C	D	E	F	G	H	I
1	#	ID	diameter	x	y	z	u	v	w
2	80	0.005	0.01171	0.625	-0.09421	-4.49E-285	1.434	0.1879	
3	61	0.005	-0.01782	0.625	-0.09415	0.01956	1.343	-1.695	
4	250	0.005	-0.01172	0.625	-0.09799	-1.01E-174	1.282	-2.726	
5	69	0.005	0.005683	0.625	-0.09588	2.95E-156	1.659	1.659	

Figure 6: Massflow raw data

Appendix A

Appropriate units for SI and CGS

SI

Mass	Kilograms
Distance	Meters
Time	Seconds
Energy	Joules
Velocity	Meters/second
Force	Newtons
Torque	Newton-meters
Temperature	Kelvin
Pressure	Pascals
Dynamic Viscosity	Pascal * second
Charge	Coulombs
Dipole	Coulomb * meters
Electric Field	Volts/meter
Density	Kilograms/meters ³

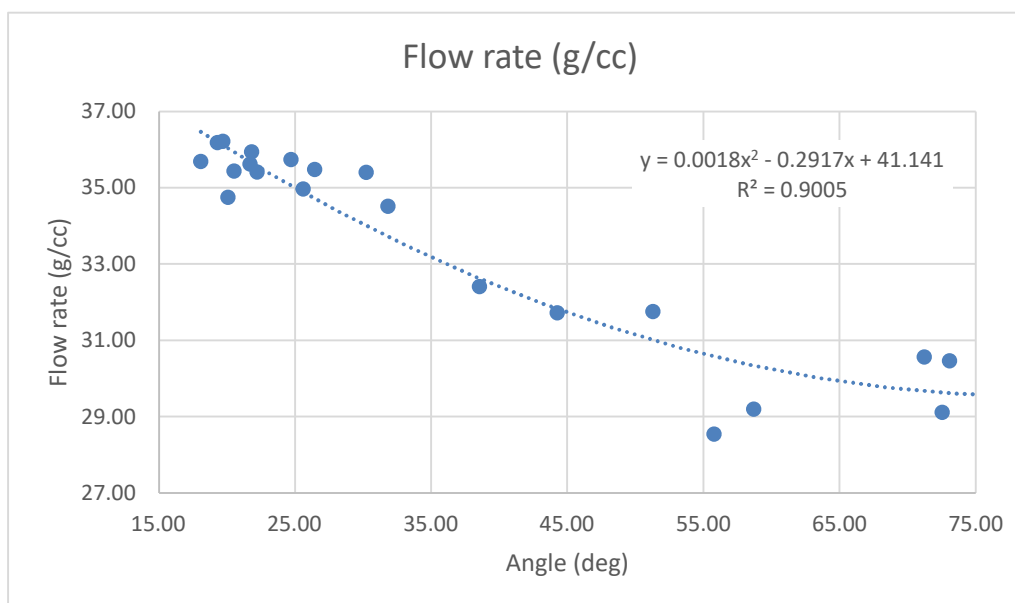
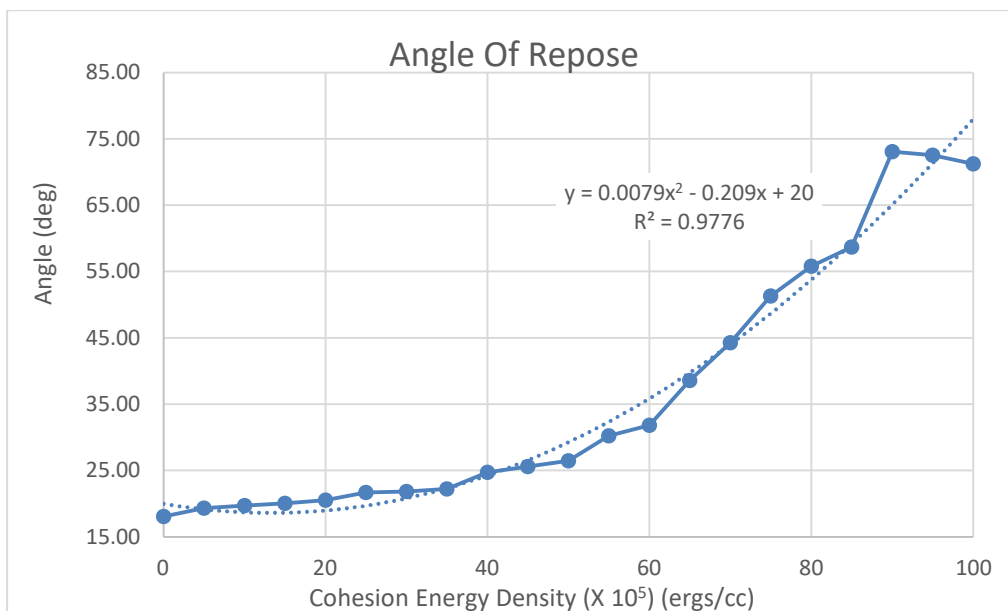
CGS

Mass	Grans
Distance	Centimeters
Time	Seconds
Energy	Ergs
Velocity	Centimeters/second
Force	Dynes
Torque	Dyne-centimeters
Temperature	Kelvin
Pressure	Dyne/cm ² (Barye = 1.6e-6 bars)
Dynamic Viscosity	Poise
Charge	Statcoulombs or esu
Dipole	Statcoul-cm (10 ¹⁸ debye)
Electric Field	Statvolt/cm (dyne/esu)
Density	Grams/cm ³

Appendix B

Results

Angle of Repose and Flow Rate With Hertz Model Results



INPUT SCRIPTS ANGLE OF REPOSE

#####Header for General

commands#####

atom_style granular #Simulation of particles

boundary f f f #fixed boundaries -> particles will be deleted if leaving the simulation box

units cgs

communicate single vel yes #default

newton off #default

hard_particles yes

```
#####System
variables#####
```

```
#Definition of boundaries
```

```
variable xmin equal -0.25
```

```
variable xmax equal 0.25
```

```
variable ymin equal -0.25
```

```
variable ymax equal 0.25
```

```
variable zmin equal -0.125
```

```
variable zmax equal 0.25
```

```
#Definition of the timestep
```

variable dt equal .5e-6
seconds.

#timestep = 0.0001 second; Each iteration step represents 0.0001

```
#####Specific variables for current
simulation#####
```

```
variable natoms equal 2 #1 -> particle #2->Inserted Geometries
```

```
####variable for material properties####
```

```
####Young Modulus####
```

```
variableyoungmodulus1 equal 5e7 #Barye
```

```
variableyoungmodulus2 equal 5e7 #
```

```
####Poisson ratio####
```

```
variable poisson1 equal 0.3
```

```
variable poisson2 equal 0.3
```

```
####variable for contact properties####
```

```
####coefficient of restitution####
```

```
variableCoR11 equal 0.2
```

```
variableCoR12 equal 0.2
```

```
variableCoR21 equal 0.2
```

```
variableCoR22 equal 0.2
```

```
####sliding friction coefficient####
```

```
variablesf11 equal 0.75
```

```
variablesf12 equal 0.75
```

```
variablesf21 equal 0.75
```

```
variablesf22 equal 0.75
```

```
####rolling friction coefficient####
```

```
variablerf11 equal 0.2
```

```
variablerf12 equal 0.7
```

```
variablerf21 equal 0.7
```

```
variablerf22 equal 0.7
```

#####Cohesion #####

variable coh11 equal 1000000

variable coh12 equal 1000000

variable coh21 equal 1000000

variable coh22 equal 1000000

#####variable for particle#####

#Number of particle radius

variablenradii equal 3

variableradius1 equal 0.002500 #cm

variableradius2 equal 0.00300#cm

variable radius3 equal 0.00100

variablefrac1 equal 1.

variablefrac2 equal 0.0

variablefrac3 equal 0.0

variabdensity equal 2.500 #g/cm³

#####Definition of
simulationbox#####

```
region          reg block ${xmin} ${xmax} ${ymin} ${ymax} ${zmin} ${zmax} units box
```

```
create_box      2 reg
```

```
neighbor        0.005 bin          #default
```

```
neigh_modify    delay 0            #default
```

```
#####Definition of the contact
models#####
```

```
pair_style gran model hertz tangential history cohesion sjkr      # the contact model used is hertz
```

```
pair_coeff       * *              #default
```

```
timestep ${dt}
```

```
fix             integrator all nve/sphere          #default
```

```
fix             gravi all gravity 981 vector 0.0 0.0 -1.0      #gravity of 981 cm/s2 in negative z
direction
```

```
#####Definition of Material
properties#####
```

```
fix             m1 all property/global youngsModulus peratomtype ${youngmodulus1}
${youngmodulus2}
```

```
fix             m2 all property/global poissonsRatio peratomtype ${poission1} ${poission2}
```

```
fix             m3 all property/global coefficientRestitution peratomtypepair ${natoms} 0.6 0.6 0.6 0.6
```

```
fix          m4 all property/global coefficientFriction peratomtypepair ${natoms} ${sf11} ${sf12}
${sf21} ${sf22}
```

```
fix          m5 all property/global coefficientRollingFriction peratomtypepair ${natoms} ${rf11}
${rf12} ${rf21} ${rf22}
```

```
fix          m6 all property/global cohesionEnergyDensity peratomtypepair ${natoms} ${coh11}
${coh12} ${coh21} ${coh22}
```

```
fix          m7 all property/global characteristicVelocity scalar 24  #this property is only used with
the hooke model, it does nothing in the hertz model
```

```
#####Generation and Loading of the Geometry
.stl#####
```

```
fix          ground all mesh/surface file Ground.stl type 2 scale 0.0001
```

```
fix          walls all wall/gran model hertz tangential no_history rolling_friction cdt mesh n_meshes 1
meshes ground      #the contact conditions of the solidworks geometry
```



```
#####Generation and Insertion of the
particles#####
```

```
fix          pts1 all particletemplate/sphere 10487 atom_type 1 density constant ${density} radius
constant ${radius1}
```

```
fix          pts2 all particletemplate/sphere 11887 atom_type 1 density constant ${density} radius
constant ${radius2}
```

```
fix          pts3 all particletemplate/sphere 11897 atom_type 1 density constant ${density} radius constant
${radius3}
```

```
fix          pdd1 all particledistribution/discrete 32452867 ${nradii} pts1 ${frac1} pts2 ${frac2} pts3
${frac3}
```

```
region factory cylinder z 0 0 0.0125 0. 0.25 units box
```

```
fix          ins all insert/rate/region seed 86028157 distributiontemplate pdd1 &
nparticles INF particlerate 30000 insert_every 1000 overlapcheck &
yes all_in yes vel constant 0 0 -.75 region factory ntry_mc 10000
```

```
thermo_style custom step atoms ke cpu
```

```
thermo 100
```

```
thermo_modify norm no lost ignore
```

```
fix ts_check all check/timestep/gran 1000 0.1 0.1
```

```
#####Dumping of the data for post-processing to  
visualize#####
```

```
shell  mkdir post
```

```
dump  dmpparticle all custom/vtk 1000 post/particles_*.vtk id type x y z vx vy vz fx fy fz radius mass
```

```
dump  dmpground all mesh/stl 1000 post/Ground*.stl ground
```

```
#####RUN the simulation
filling#####
run 1
run      450000

unfix ins

run 150000
```

INPUT SCRIPTS FOR TROUGH FEEDER

```
#####trough feeder with vibration#####
#####6/26/2020 safwat shenouda#####
#####
#This simulation is for the vibratory feeder, at 113 Hz
```

#####standard box- to be used for practically all simulations#####

atom_style granular #can either be granular or sphere

atom_modify map array #optional command

boundary f f f #bounding box is fixed and doesn't move. particles will be deleted when
they leave the bounding box

units cgs #units are centimeters/grams/seconds

communicate single vel yes #standard comment, but can be changed depending on cores and
parrallelization

newton off #standard comment for every simulation

hard_particles yes #overrides limit on youngs modulus, use soft_particles yes for rubbery
materials

```
#####boundary variables#####
```

```
variable xmin equal -.125
```

```
#Definition of boundary box all units in cm
```

```
variable xmax equal .125
```

```
variable ymin equal -0.0125
```

```
variable ymax equal 0.625
```

```
variable zmin equal -.125
```

```
variable zmax equal 0.125
```

```
#####timestep#####
```

```
variable dt equal .5e-6 # ~15% of rayleigh timestep
```

#####material properties#####

#####youngs modulus #####

variable modulus1 equal 5e7 #barye, for particle

variable modulus2 equal 5e7 #for geometry

#####poissons ratio #####

variable poisson1 equal 0.3 #poissons ratio of particle

variable poisson2 equal 0.3 #poissons ratio of geometry

#####variables for particles#####

#####number of particle types#####

variable nradii equal 3 #number of different particles

#####contact properties - can be calibrated through experimentation or other
methods#####

variable nobj equal 2 #how many objects there are. In this case, there is 1
particle type and 1 geometric type

variable cor11 equal 0.2 #coefficient of restitution particle-to-particle

variable cor12 equal 0.2 #coefficient of restitution particle-to-geometry

variable cor21 equal 0.2 #coefficient of restitution geometry-to-particle

variable cor22 equal 0.2 #coefficient of restitution geometry-to-geometry

variable sf11 equal 0.75 #sliding friction coefficient particle-to-particle

variable sf12 equal 0.75 #sliding friction coefficient particle-to-geometry

variable sf21 equal 0.75 #sliding friction coefficient geometry-to-particle

```

variable sf22      equal 0.75      #sliding friction coefficient geometry-to-geometry
variable rf11      equal 0.2       #rolling friction coefficient particle-to-particle
variable rf12      equal 0.7       #rolling friction coefficient particle-to-geometry
variable rf21      equal 0.7       #rolling friction coefficient geometry-to-particle
variable rf22      equal 0.7       #rolling friction coefficient geometry-to-geometry
variable coh11     equal 900000
variable coh12     equal 900000
variable coh21     equal 900000
variable coh22     equal 900000

```

```
#####particle distribution#####
```

```

variableradius1    equal 0.0025    #centimeters
variable radius2    equal 0.0030
variableradius3     equal 0.0025
variable frac1      equal 1.0       #particle size distribution
variablefrac2       equal 0.0
variablefrac3       equal 0.0
variabledensity     equal 2.5       #density g/cm^3

```



```
#####bounding box - should be minimized for efficiency#####
```

```
region          reg block ${xmin} ${xmax} ${ymin} ${ymax} ${zmin} ${zmax} units box
                #command for inserting a region, type is block for rectangular grid
```

```
create_box 2 reg
            #required command for this type of region
```

```
#####neighbors standard parameters, can be changed to optimize performance
```

```
neighbor 0.005 bin      #usually set to the center to center distance between 2 particles
```

```
neigh_modify delay 0
```

```
#####gravity#####
```

```
pair_style gran model hertz tangential history cohesion sjkr          # the contact model used is
hertz
```

```
pair_coeff * *
```

```
variable deg equal 3          #3 degrees for the angle of lincline
```

```
variable rad equal ${deg}*PI/180      #converting degrees to radians
```

```
variable y equal sin(${rad})          #y vector component
```

```
variable z equal cos(${rad})          #z vector component
```

```
fix gravi all gravity 981 vector 0.0 -$y -$z      #gravitational constant
```

```
#####integration style
```

```
timestep ${dt}          #where timestep is determined
```

```
fix ts_check all check/timestep/gran 1000 0.1 0.1      #checks timestep and outputs results in terminal
```

```
fix integr all nve/sphere      #multisphere and superquadratic are other options
```

```
#####setup#####
```

```
#material properties
```

```
fix m1 all property/global youngsModulus peratomtype ${modulus1} ${modulus2}      # this is how  
you reference a previously defined variable
```

```
fix m2 all property/global poissonsRatio peratomtype ${poisson1} ${poisson2}
```

```
fix m3 all property/global coefficientRestitution peratomtypepair ${nobj} ${cor11} ${cor12} ${cor21}  
${cor22}
```

```
fix m4 all property/global coefficientFriction peratomtypepair ${nobj} ${sf11} ${sf12} ${sf21} ${sf22}
```

```
fix m5 all property/global coefficientRollingFriction peratomtypepair ${nobj} ${rf11} ${rf12} ${rf21}  
${rf22}
```

```
fix m6 all property/global cohesionEnergyDensity peratomtypepair ${nobj} ${coh11} ${coh12}  
${coh21} ${coh22}
```

```
#####
```

```
#####Loading Geometry#####
```

```
#####
```

```
fix cad1 all mesh/surface file Wiggle.stl type 2 scale 0.01      # this is where the solidworks model is
inserted from as an stl file. Note that windows uses .STL and linux used .stl.
```

```
fix walls all wall/gran model hertz tangential no_history rolling_friction cdt mesh n_meshes 1 meshes
cad1      #the contact conditions of the solidworks geometry
```

```
#####how massflow is calculated#####
```

```
#####importing plane for massflow calculation
```

```
#####
```

```
fix plane1 all mesh/surface/planar file massface.stl type 2 scale 0.01
```

```
fix mass all massflow/mesh mesh plane1 count once delete_atoms no vec_side 0. 0.625 0. file
massflow.txt      #creates the raw data file
```

```
#####
#####particle creation and insertion#####
#####

# this lines are standard boxes for particle insertion. the numbers used are for particle insertion purposes
and need to be larger than 10000 and must be prime numbers

fix pts1 all particletemplate/sphere 10487 atom_type 1 density constant ${density} radius constant
${radius1}      #add more of these lines for making a particle distribution

fix pts2 all particletemplate/sphere 11887 atom_type 1 density constant ${density} radius constant
${radius2}

fix pts3 all particletemplate/sphere 11897 atom_type 1 density constant ${density} radius constant
${radius3}

fix pdd1 all particledistribution/discrete 32452867 ${nradii} pts1 ${frac1} pts2 ${frac2} pts3 ${frac3}
      #add the new fix ID and the distribution fraction

region factory cylinder z 0 .2 0.01 -0.05 0. units box      #creates the insertion region, a cylinder shaped
object z, x, y radius, lower bound, upper bound

fix      ins all insert/rate/region seed 86028157 distributiontemplate pdd1 &
      nparticles INF particlerate 10000 insert_every 1000 overlapcheck &
      yes all_in yes vel constant 0 0 -.1 region factory ntry_mc 10000 # the '&' symbol restarts
a line on the next line
```

```
compute 1 all erotate/sphere          #output of thermodynamic data
thermo_style custom step atoms ke cpu
thermo 100
thermo_modify norm no lost ignore      #ignores particles exiting the boundary
```

```
#####
#####creation of dump files#####
#####
```

#create imaging info. this command creates a dump folder containing the simulation files for the particles, variables, and geometries to be used in post-processing

```
shell  mkdir post    #creating a folder named 'post'
```

```
dump dmpparticle all custom/vtk 1150 post/particles_*.vtk id type x y z vx vy vz fx fy fz ix iy iz omegax  
omegay omegaz radius mass #dumps a file every 1150 steps
```

```
dump dmpWiggle all mesh/stl 1150 post/Wiggle*.stl #dumps a file every 1150 steps
```

```
#####
```

```
#####moving the stl geometry (trough)#####
```

```
fix      move all move/mesh mesh cad1 wiggle amplitude 0.0 0.0025 0.0025 period 0.0088496  
#vibration of trough x,y,z amplitude vector, and period
```

```
#####running the simulation#####
```

```
run 150000 #runs the simulation 150000 steps
```

unfix ins #stops the insertion of particles at 150000

run 1000000 #simulation continues to run 1000000 more steps

INPUT SCRIPTS FOR BOUNCING BALL

1. Simulation pre-processing & definition of variables

These parameters are the 1st step in beginning any LIGGGHTS simulation. They define the particle type, processing parameters, and unit system. During the pre-processing phase, any variables will be defined so that altering these variables later to form different simulations will be simple.

<u>atom style</u>	granular
<u>atom modify</u>	map array
<u>boundary</u>	f f f
<u>newton</u>	off
<u>communicate</u>	single vel yes
<u>units</u>	si
<u>hard particles</u>	yes

a. Definition of boundary box and time-step variables

This is where the simulation bounding box is defined. The minimum and maximum values of the x, y, and z coordinates are defined. It may be needed to slightly increase each values to reduce any error due to any external geometries.

<u>variable</u>	xmin	equal	-1.0
<u>variable</u>	xmax	equal	1.0
<u>variable</u>	ymin	equal	-1.0
<u>variable</u>	ymax	equal	1.0
<u>variable</u>	zmin	equal	0.0
<u>variable</u>	zmax	equal	6.0001
<u>variable</u>	dt	equal	1e-4

b. Material properties and other variables

Here is the definition of any material properties for the inserted particles and geometry meshes used. The coefficient of restitution, sliding friction, and rolling coefficient include values for contact properties and need to have separate definitions for each interaction, such as particle-to-particle, particle-to-geometry, geometry-to-geometry, and geometry-to-particle. In this model, friction is ignored and there is no geometrical meshes to include in the simulation.

<u>variable</u>	natoms	equal	1
<u>variable</u>	youngmodulus1	equal	200.e9
<u>variable</u>	poisson1	equal	0.3
<u>variable</u>	CoR1	equal	0.9
<u>variable</u>	sf1	equal	0.0
<u>variable</u>	rf1	equal	0.0

c. Particle parameters

This is where the variables for the particles are defined. The particle size, size distribution, and particle density are given below. This section is specifically important when inserting multiple different particles with varying properties.

<u>variable</u>	nradii	equal	1
<u>variable</u>	radius11	equal	0.0
<u>variable</u>	frac1	equal	1.0
<u>variable</u>	density1	equal	7800

2. Material properties and additional pre-processing steps

Now that all the variables are defined, it is time to create the material properties and complete the rest of the pre-processing phase using our variables.

a. Boundary creation

<u>region</u>	reg block \${xmin} \${xmax} \${ymin} \${ymax} \${zmin} \${zmax} units box
<u>create box</u>	1 reg
<u>neighbor</u>	0.001 bin
<u>neigh modify</u>	delay 0

b. Material properties

<u>fix</u>	m1 all	property/global youngsModulus peratomtype	\${youngmodulus1}
<u>fix</u>	m2 all	property/global poissonsRatio peratomtype	\${poisson1}
<u>fix</u>	m3 all	property/global coefficientRestitution peratomtypepair	\${natoms} \${CoR1}
<u>fix</u>	m4 all	property/global coefficientFriction peratomtypepair	\${natoms} \${sf1}
<u>fix</u>	m5 all	property/global coefficientRollingFriction peratomtypepair	\${natoms} \${rf1}

3. Pair styles, integration , and gravitational constant

<u>pair_style</u>	gran model hertz tangential history rolling_friction epsd2
<u>pair_coeff</u>	* *
<u>timestep</u>	\${dt}
<u>fix</u>	1 all nve/sphere
<u>fix</u>	2 all gravity 9.81 vector 0.0 0.0 -1.0
<u>variable</u>	zpos equal xcm(all,z)

4. Definition of thermodynamic properties, atom creation, and location of atom drop

<u>thermo_style</u>	custom step atoms cpu spcpu v_zpos
<u>thermo</u>	100
<u>thermos_modify</u>	lost ignore norm no
<u>create_atoms</u>	1 single 0 0 1.5 units box
<u>group</u>	tmp_group id ==1
<u>set</u>	group tmp_group density \${density1}

5. Running simulation and dumping of files for post-processing

<u>fix</u>	boxwall_z all wall/gran model hertz tangential history primitive type 1 zplane 0.0
<u>dump</u>	dmp all custom 100 post/dump*.liggghts id type x y z vx vy vz fx fy fz radius mass
<u>run</u>	100000 upto

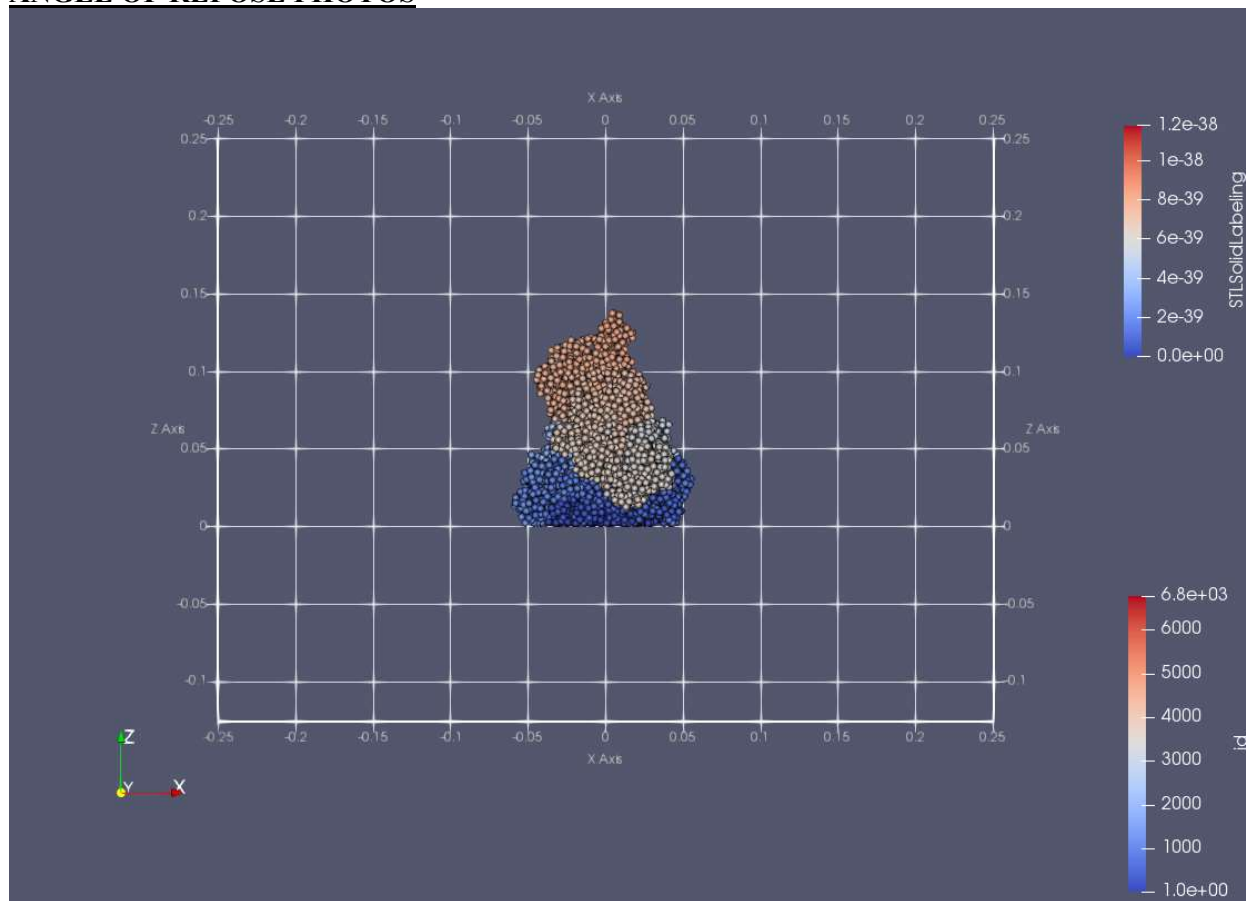
ANGLE OF REPOSE PHOTOS

Figure 7: CED = 950000 ergs/cc, 72.54 deg

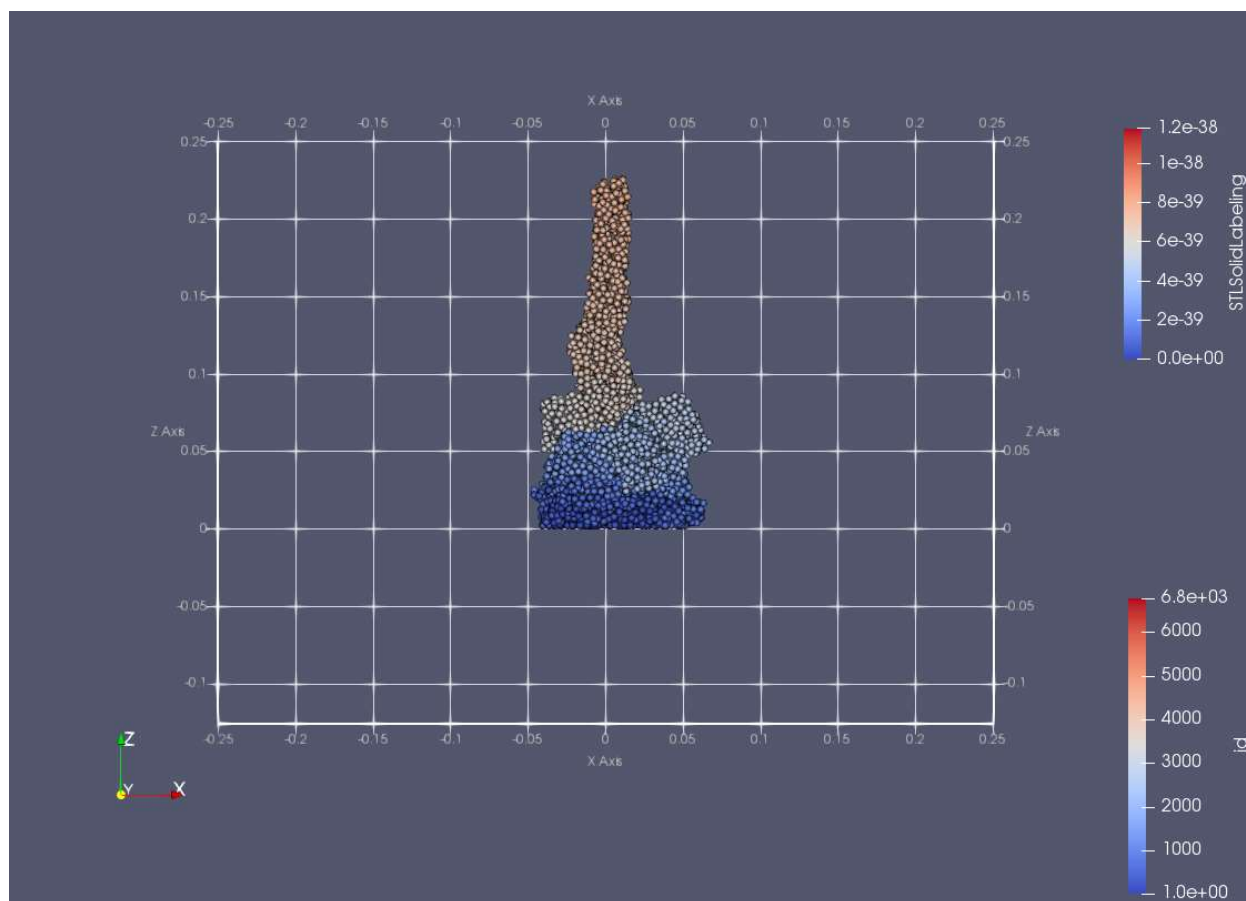


Figure 8: CED = 900000 ergs/cc, 73.08 deg

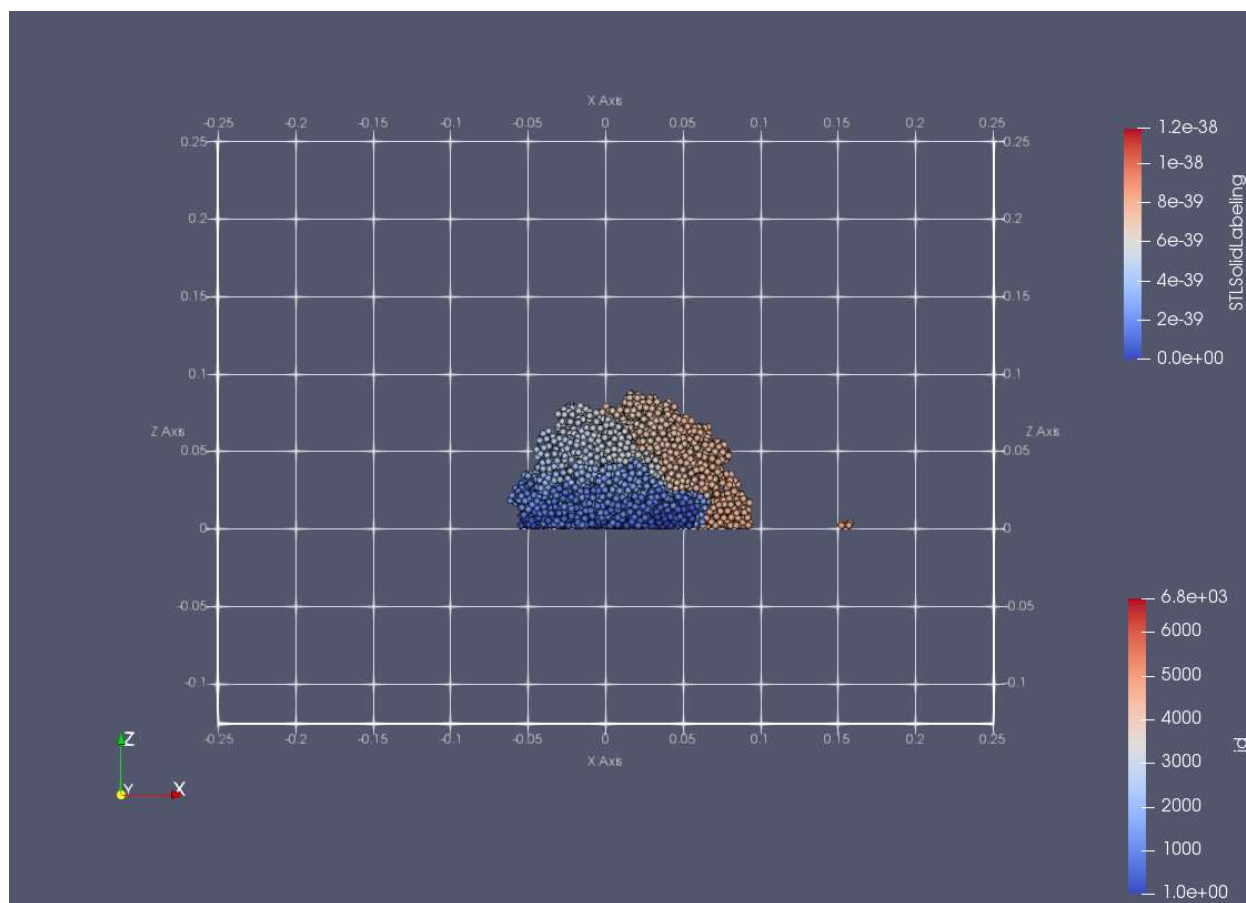


Figure 9: CED = 850000 ergs/cc, 58.7 deg

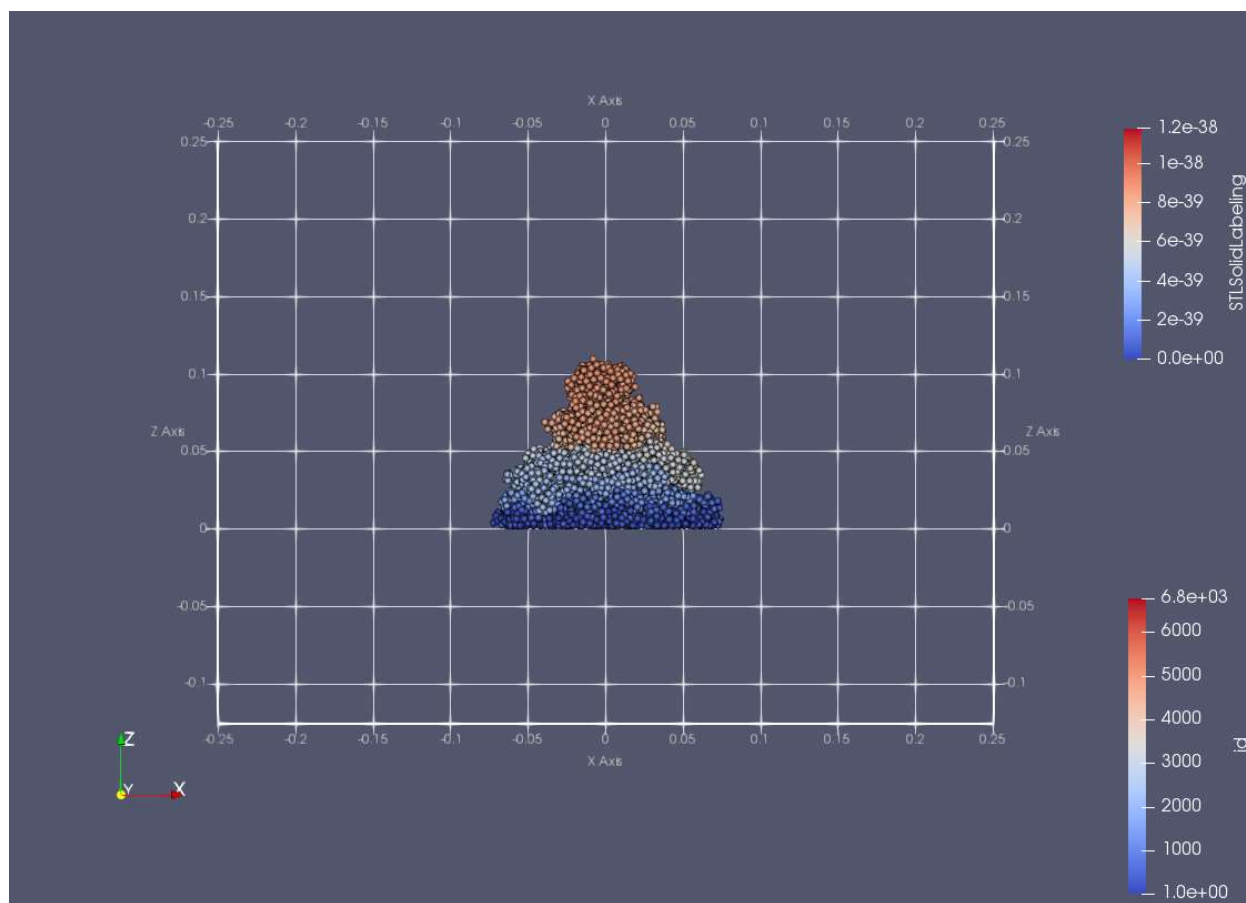


Figure 10: CED = 800000 ergs/cc, 55.79 deg

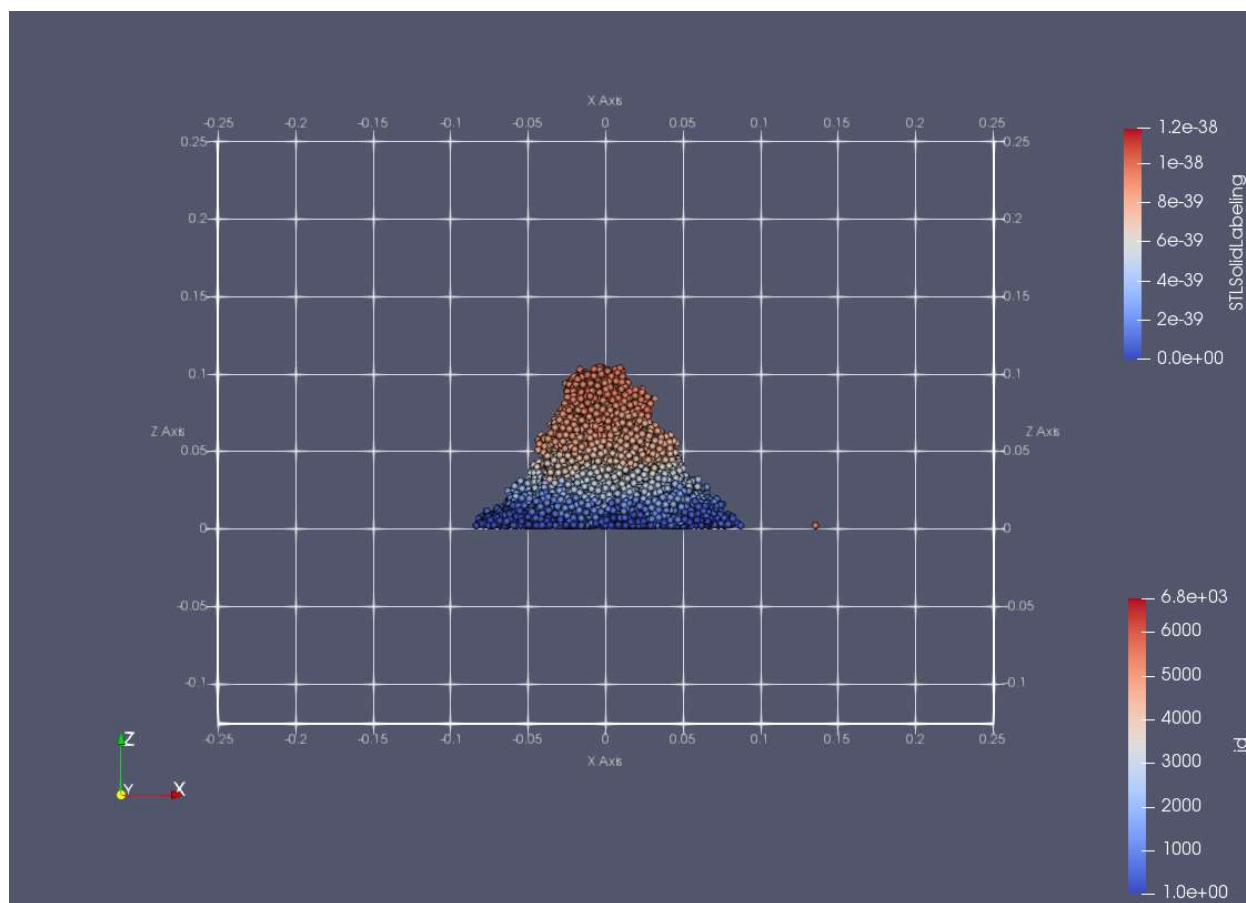


Figure 11: CED = 750000 ergs/cc, 51.29 deg

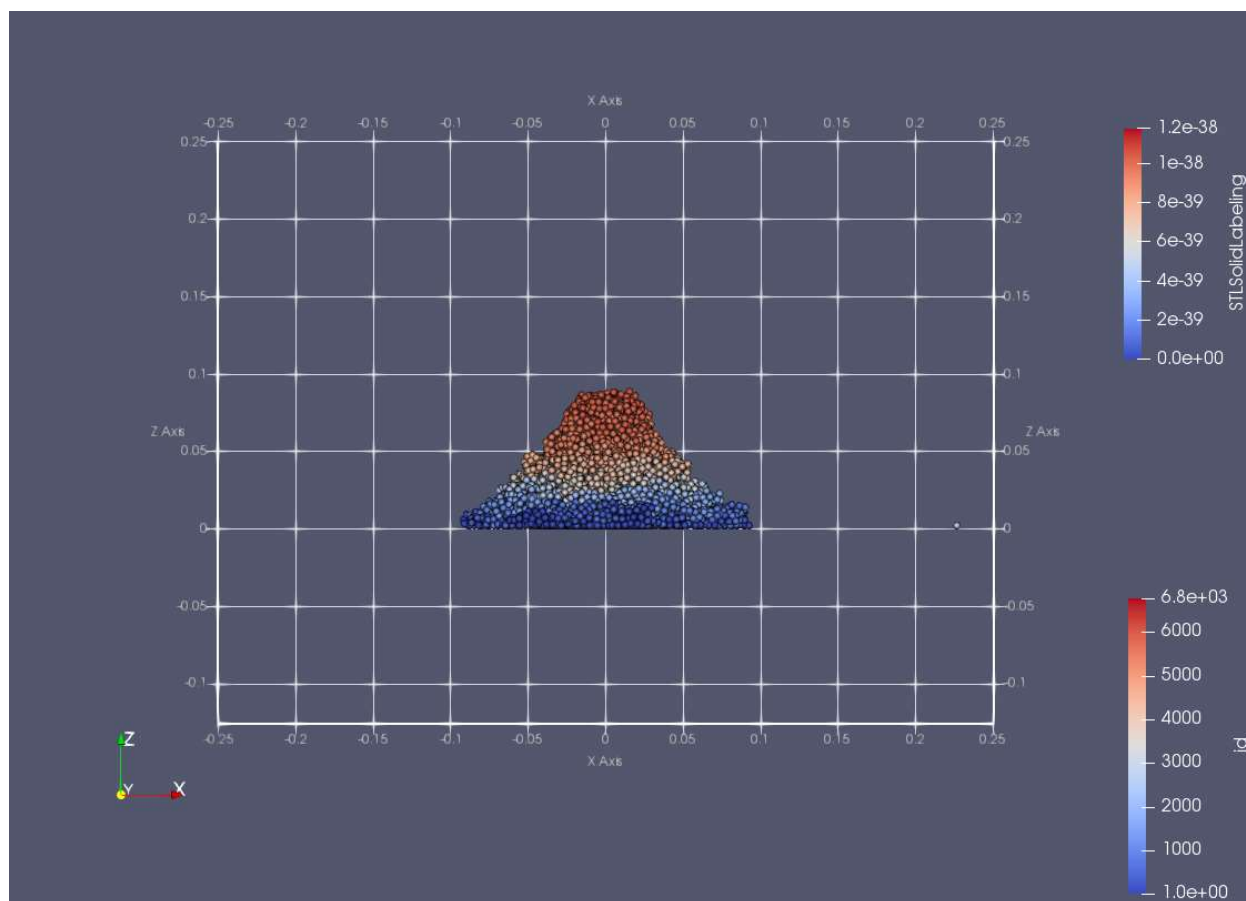


Figure 12: CED = 700000ergs/cc, 44.26 deg

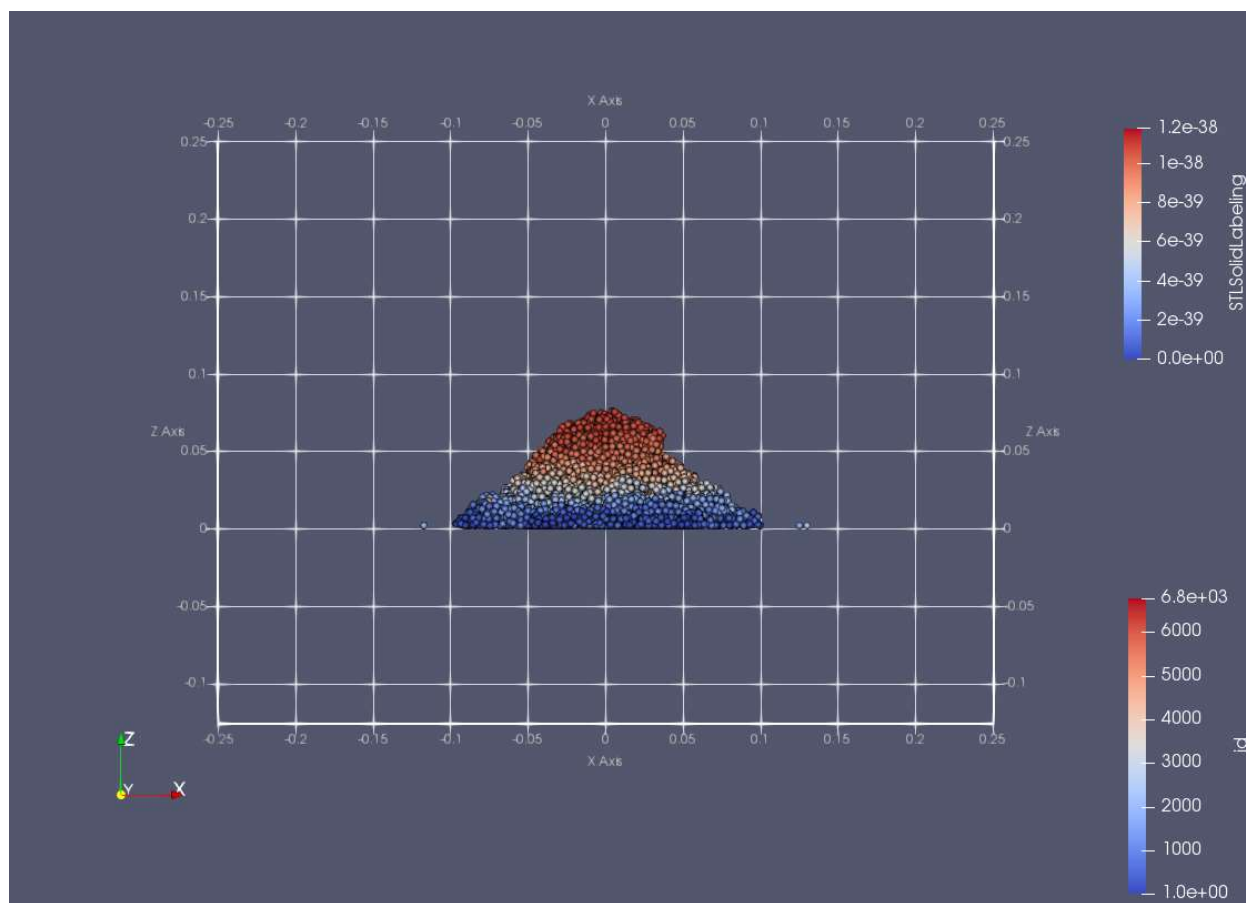


Figure 13: CED = 650000 ergs/cc, 38.55 deg

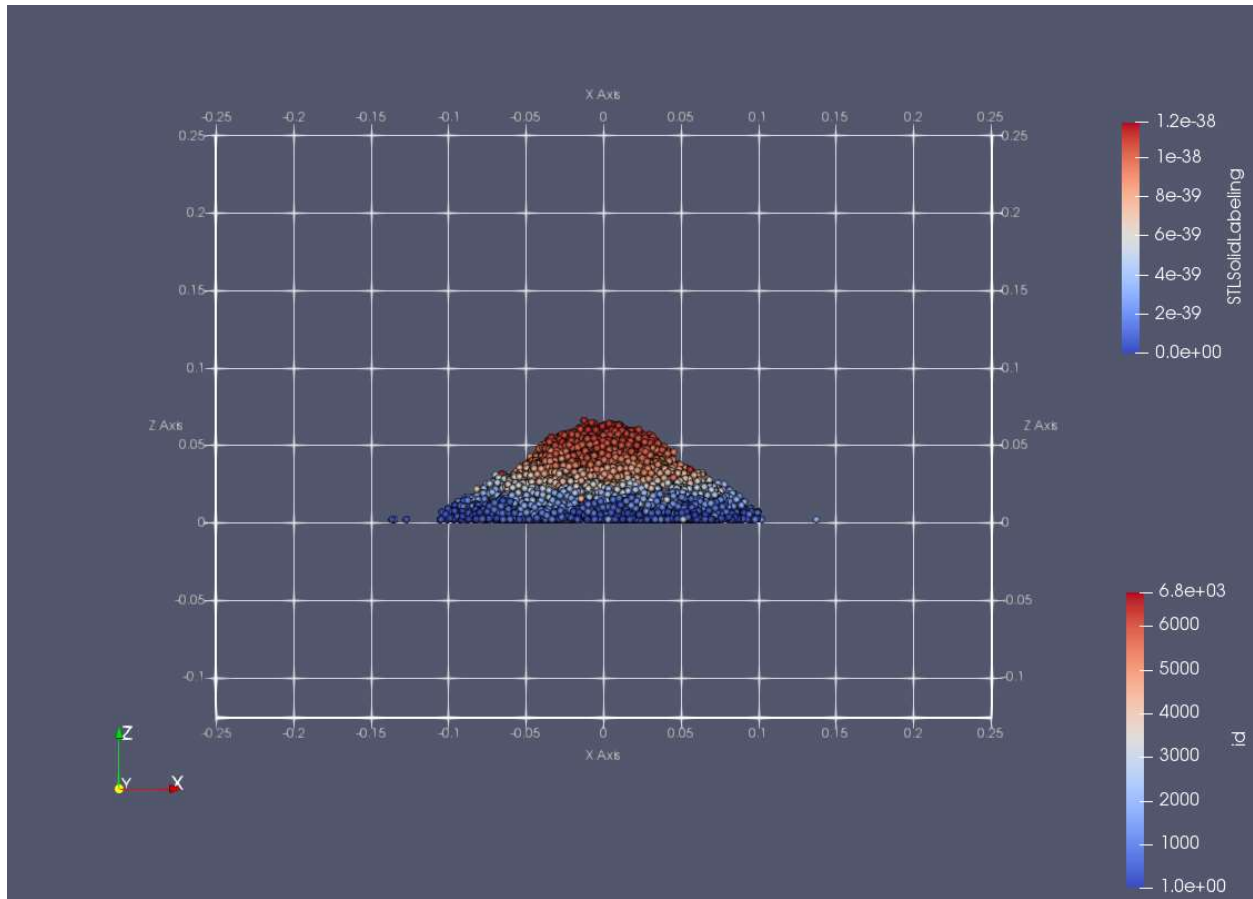


Figure 14: CED = 600000 ergs/cc, 31.83 deg

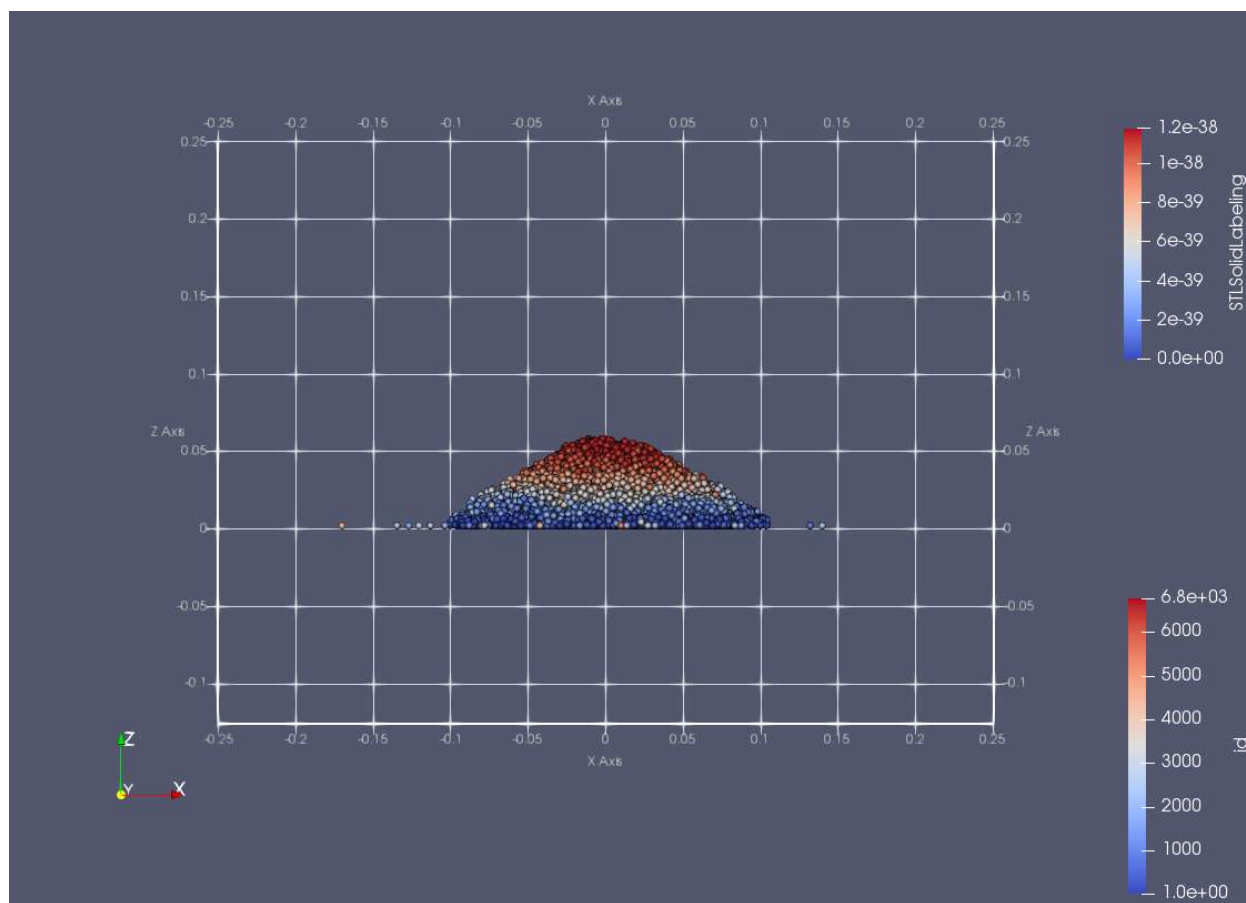


Figure 15: CED = 550000 ergs/cc, 30.22 deg

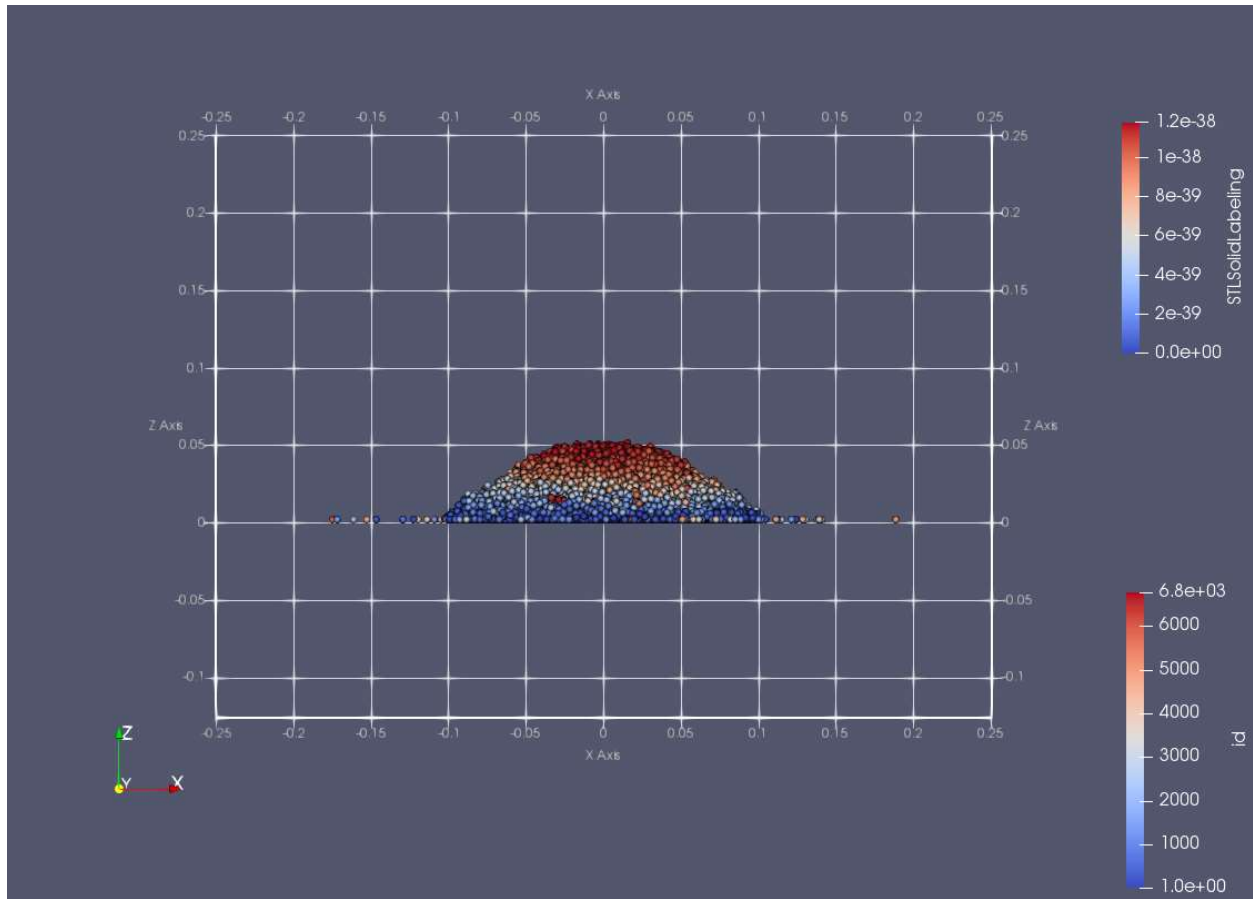


Figure 16: CED = 500000 ergs/cc, 26.45 deg

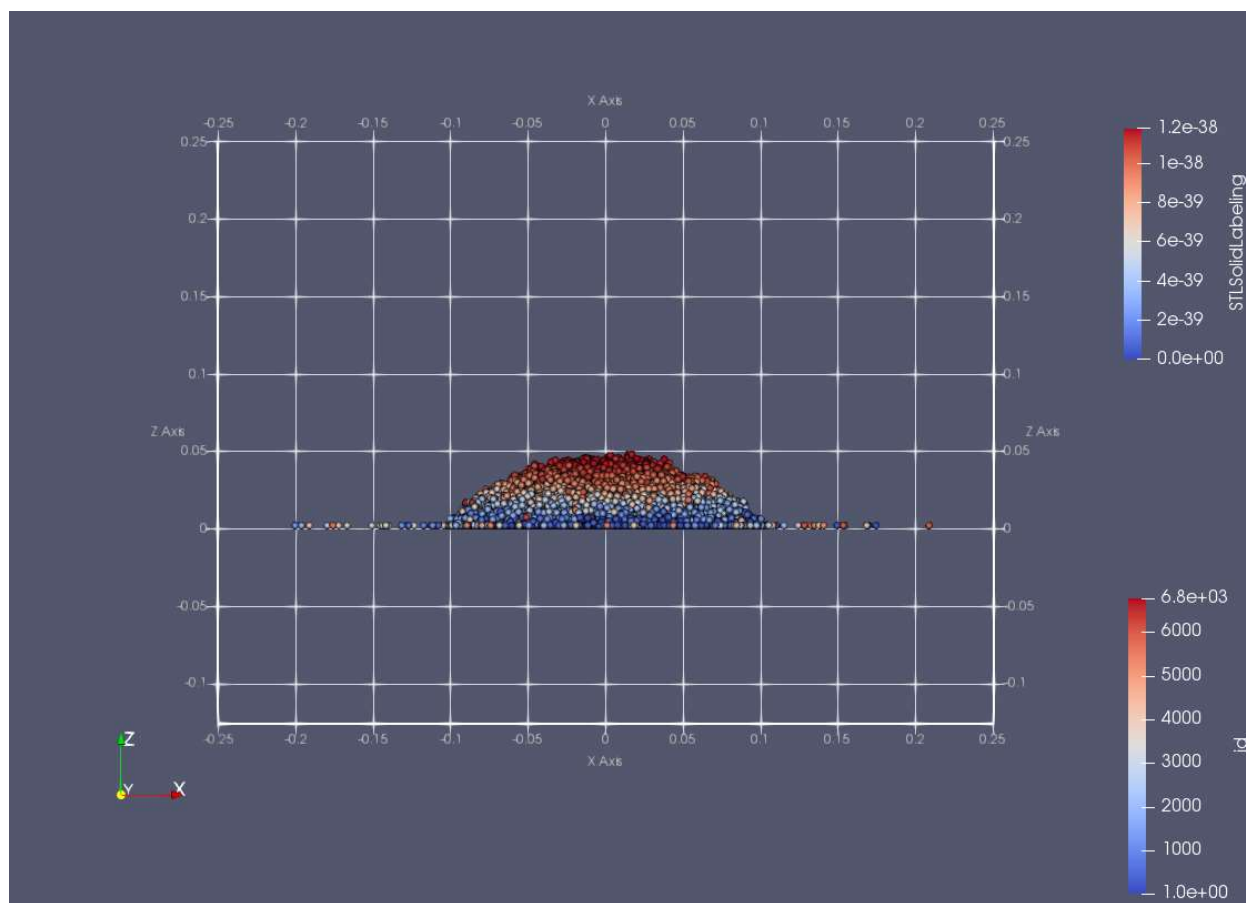


Figure 17: CED = 450000 ergs/cc, 25.59 deg

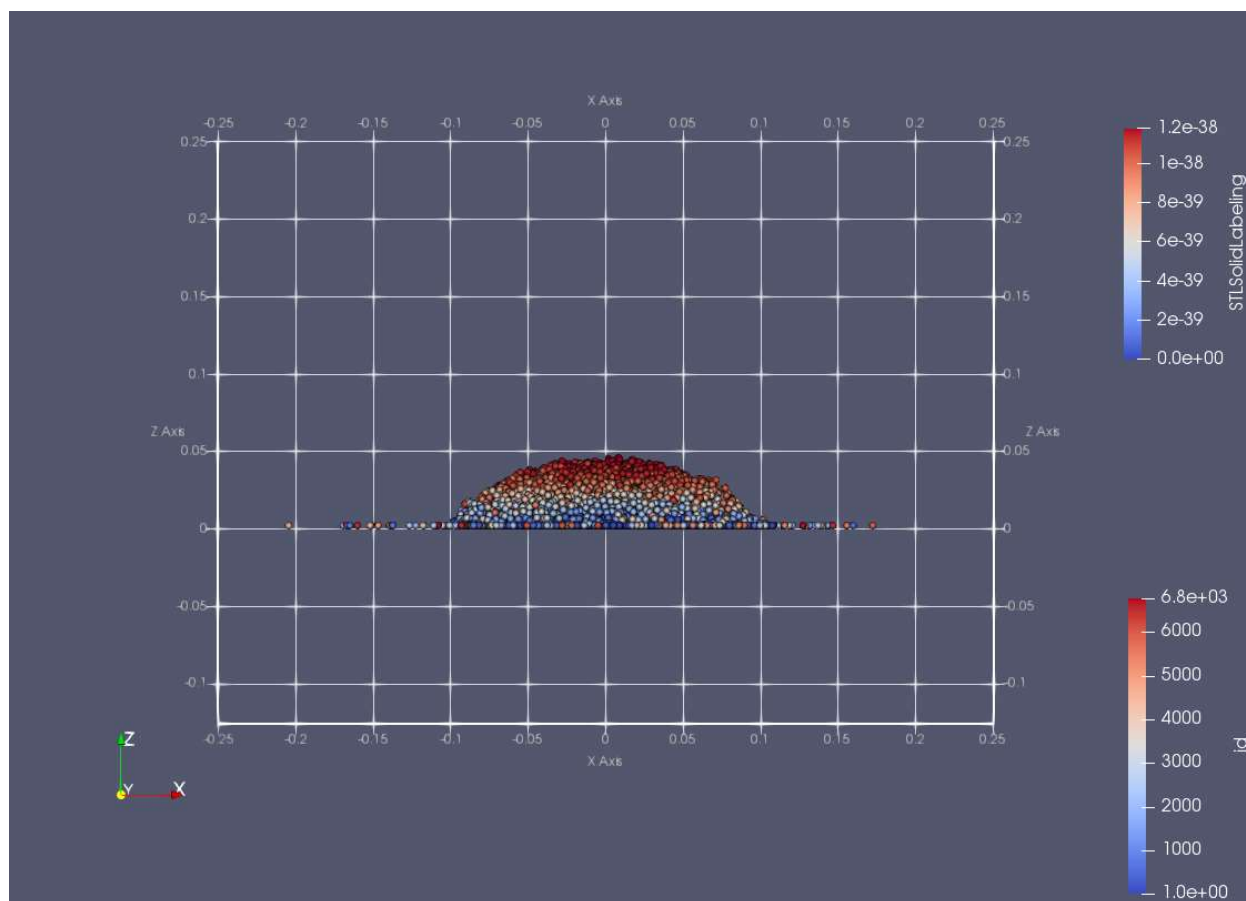


Figure 18: 400000 ergs/cc, 24.7 deg

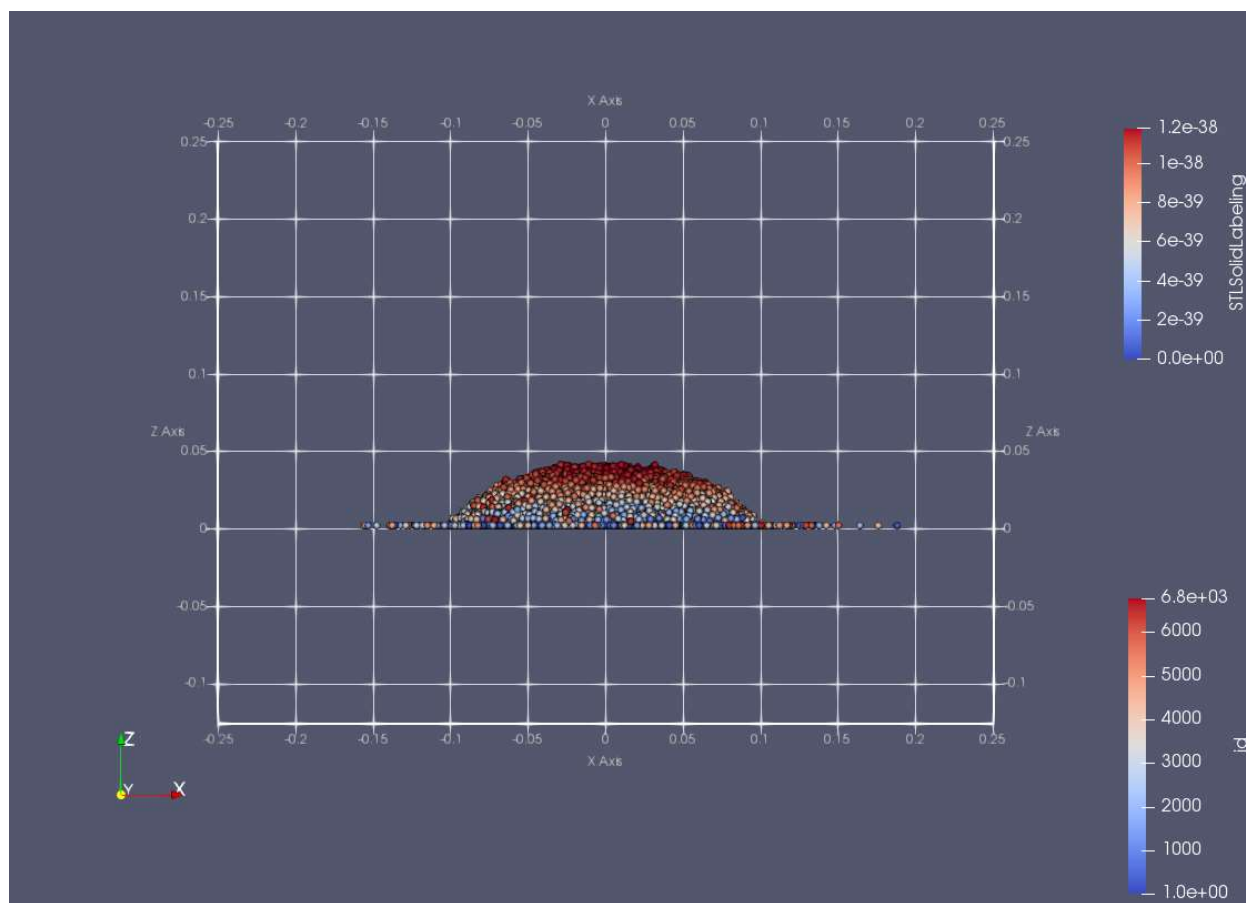


Figure 19: CED = 350000 ergs/cc, 22.22 deg

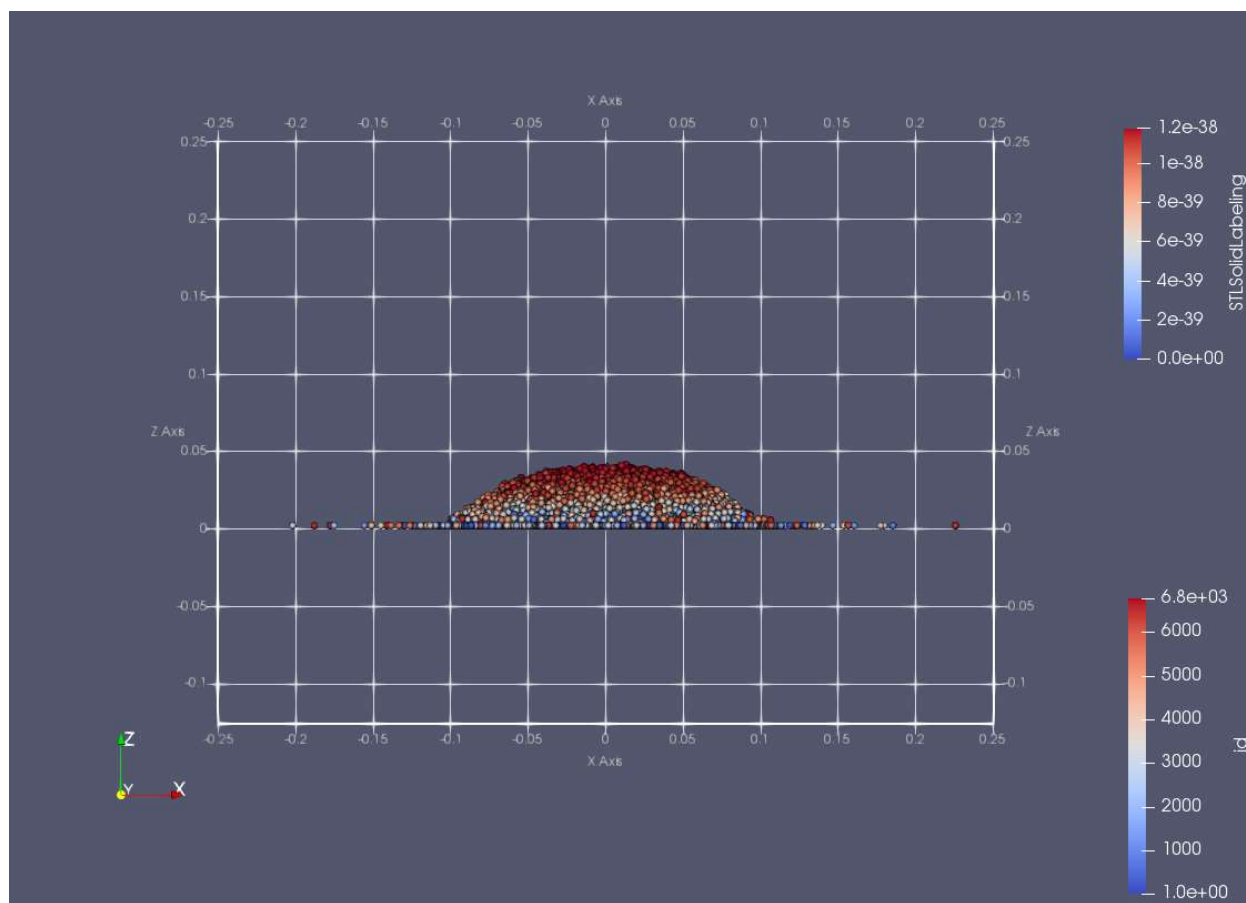


Figure 20: CED = 300000 ergs/cc, 21.82 deg

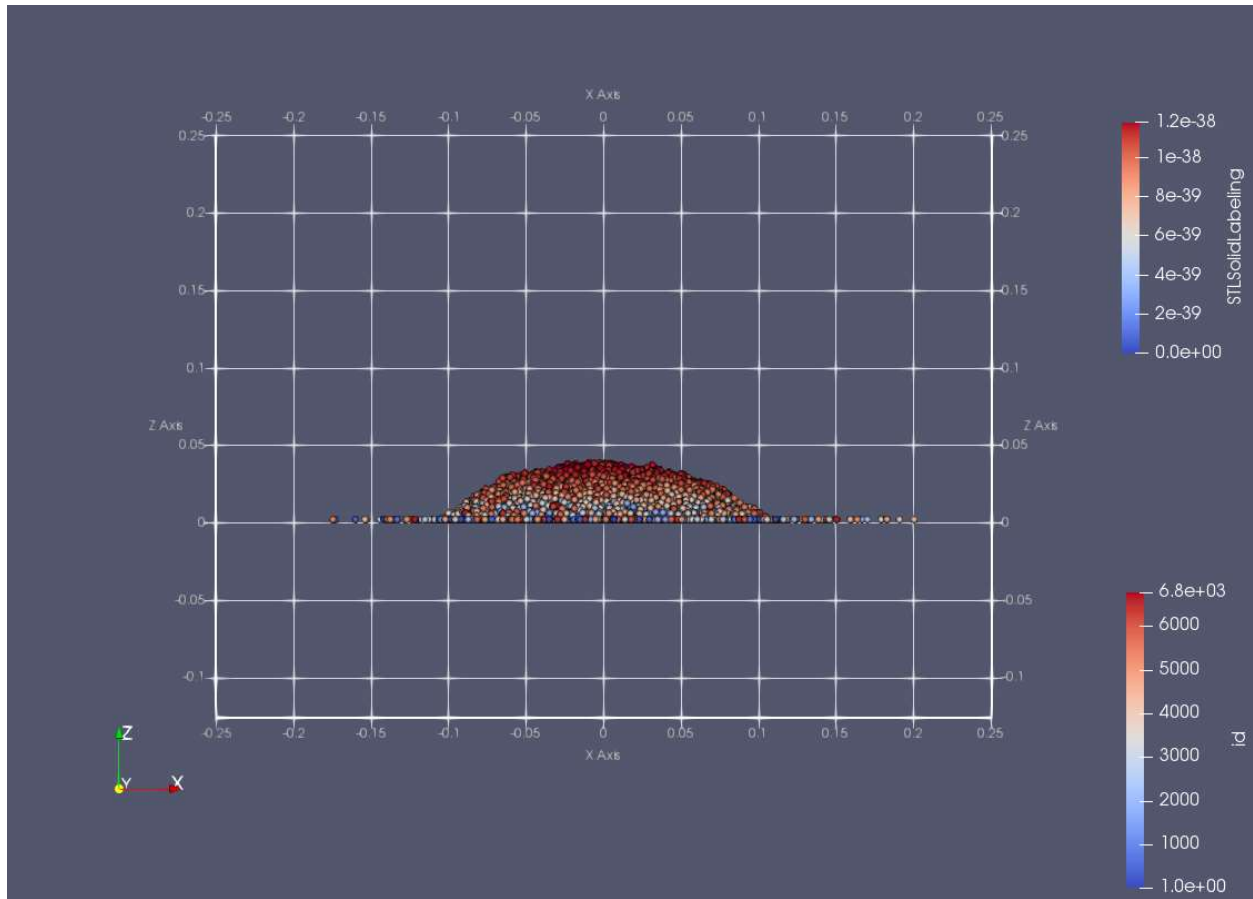


Figure 21: CED = 250000 ergs/cc, 21.69 deg

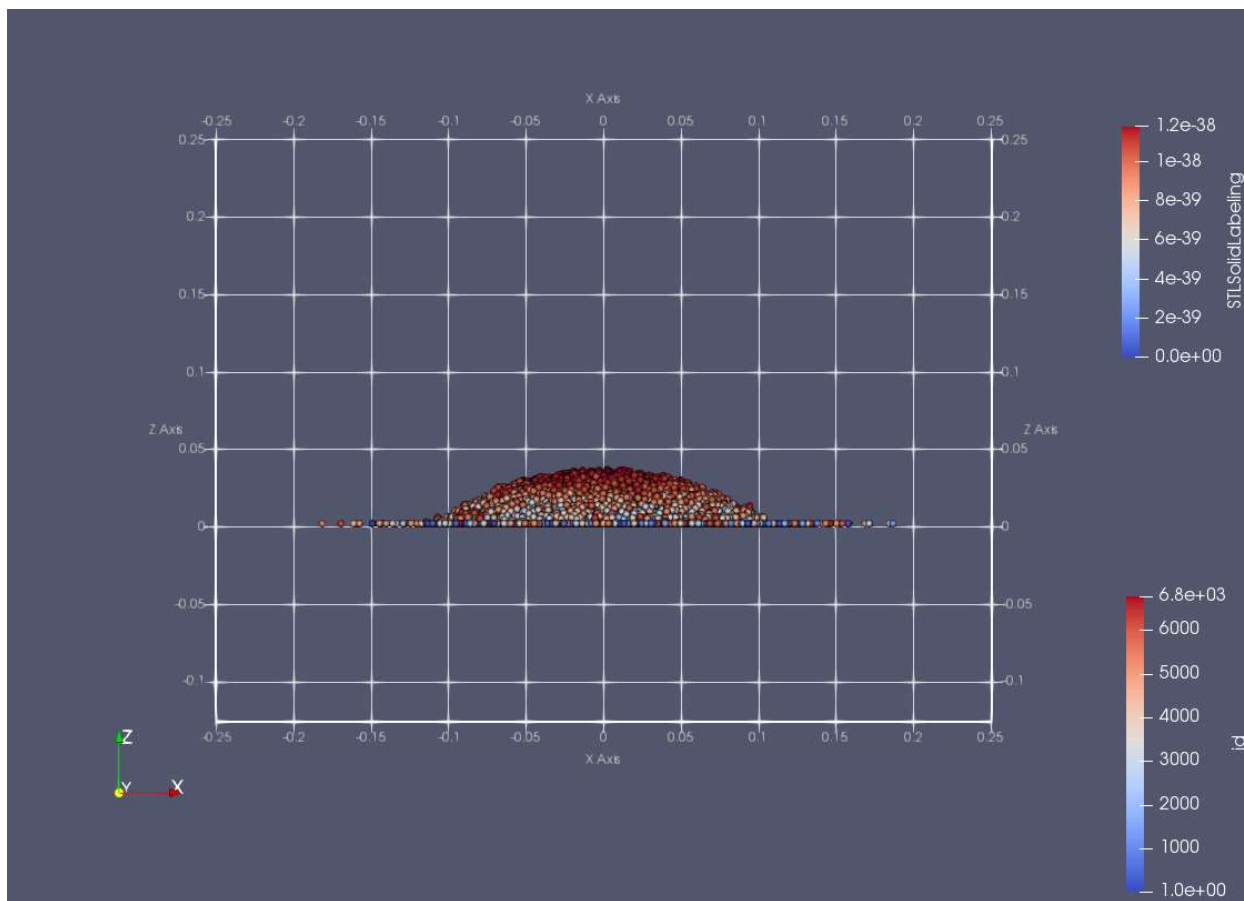


Figure 22: CED = 200000 ergs/cc, 20.52 deg

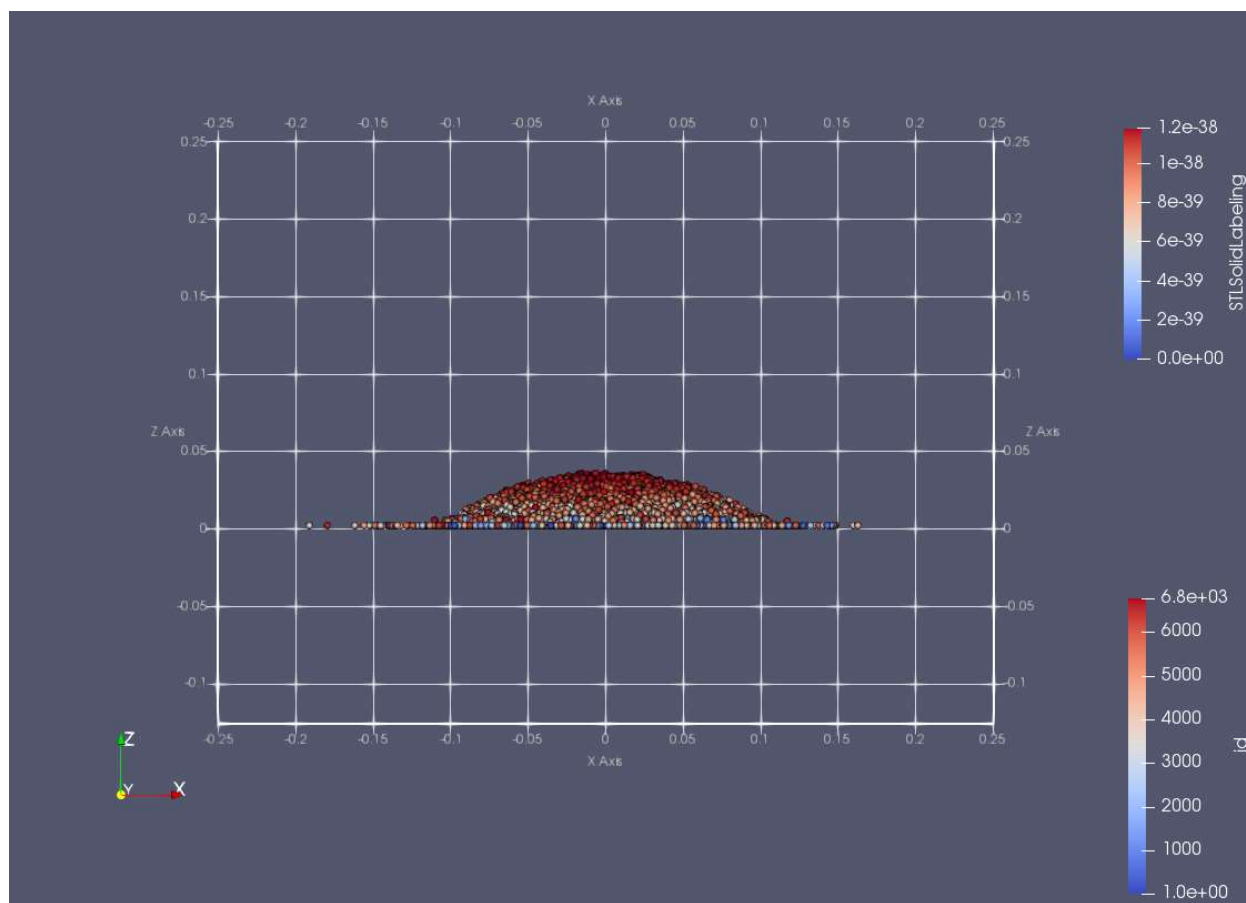


Figure 23: CED = 150000 ergs/cc, 20.06 deg

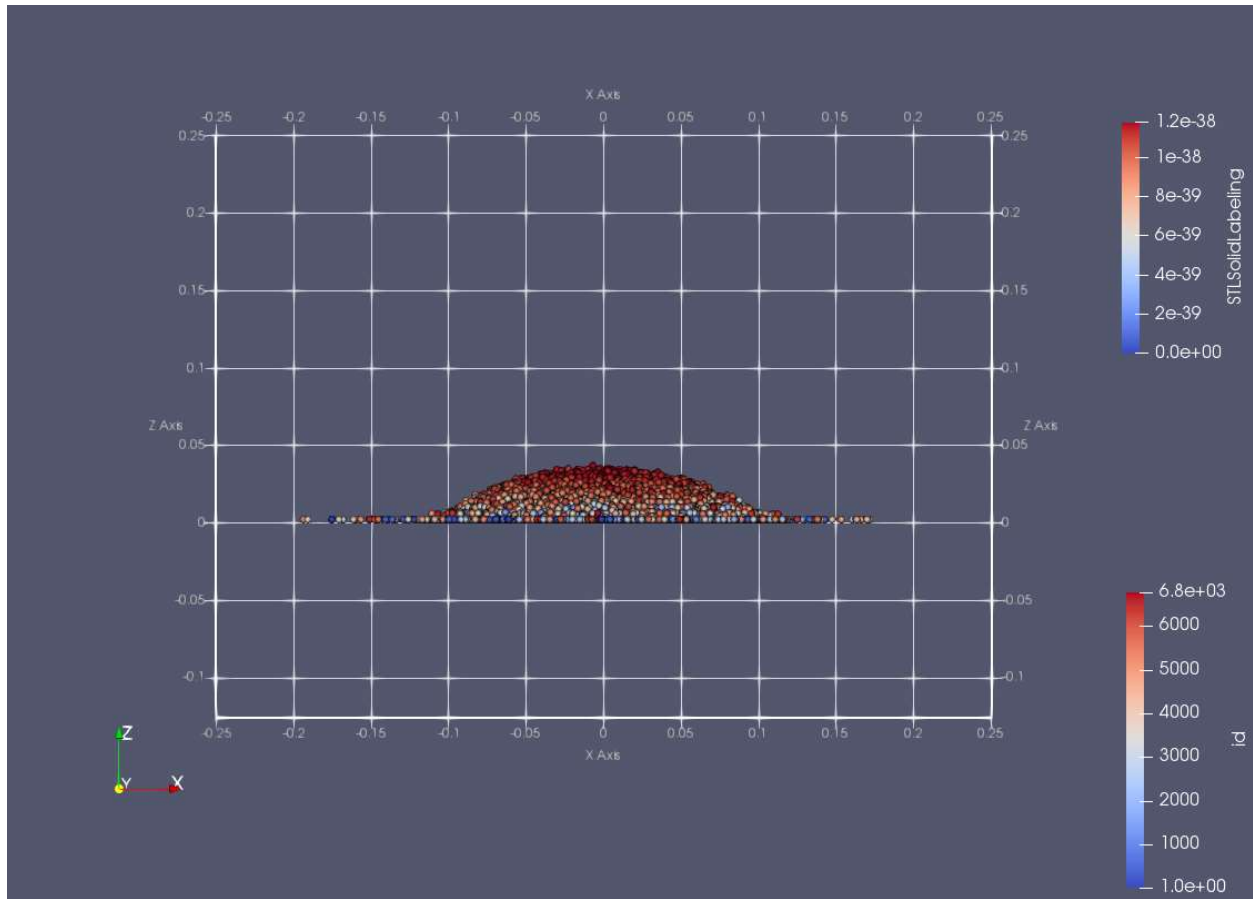


Figure 24; CED = 100000 ergs/cc, 19.7 deg

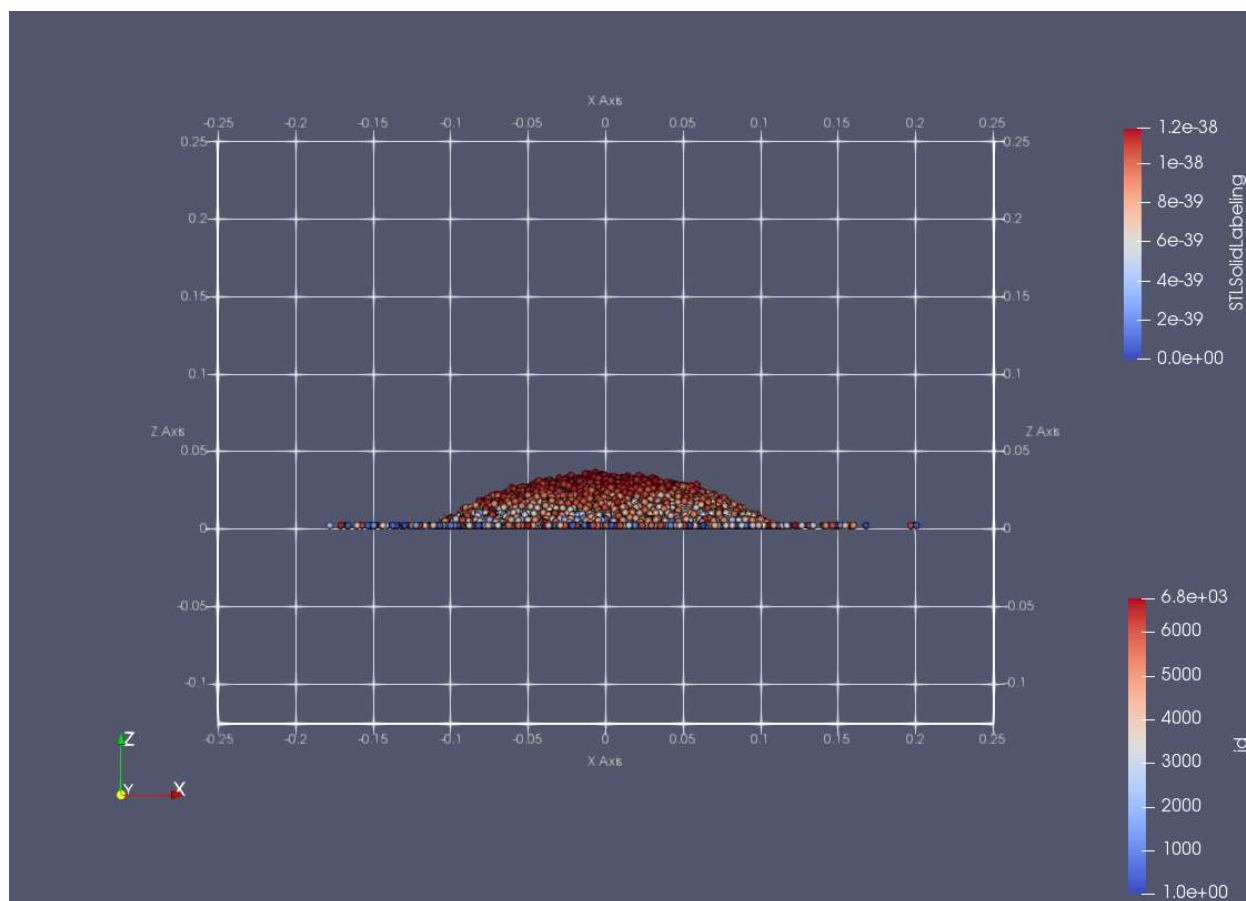


Figure 25: CED = 5000 ergs/cc, 19.30 deg

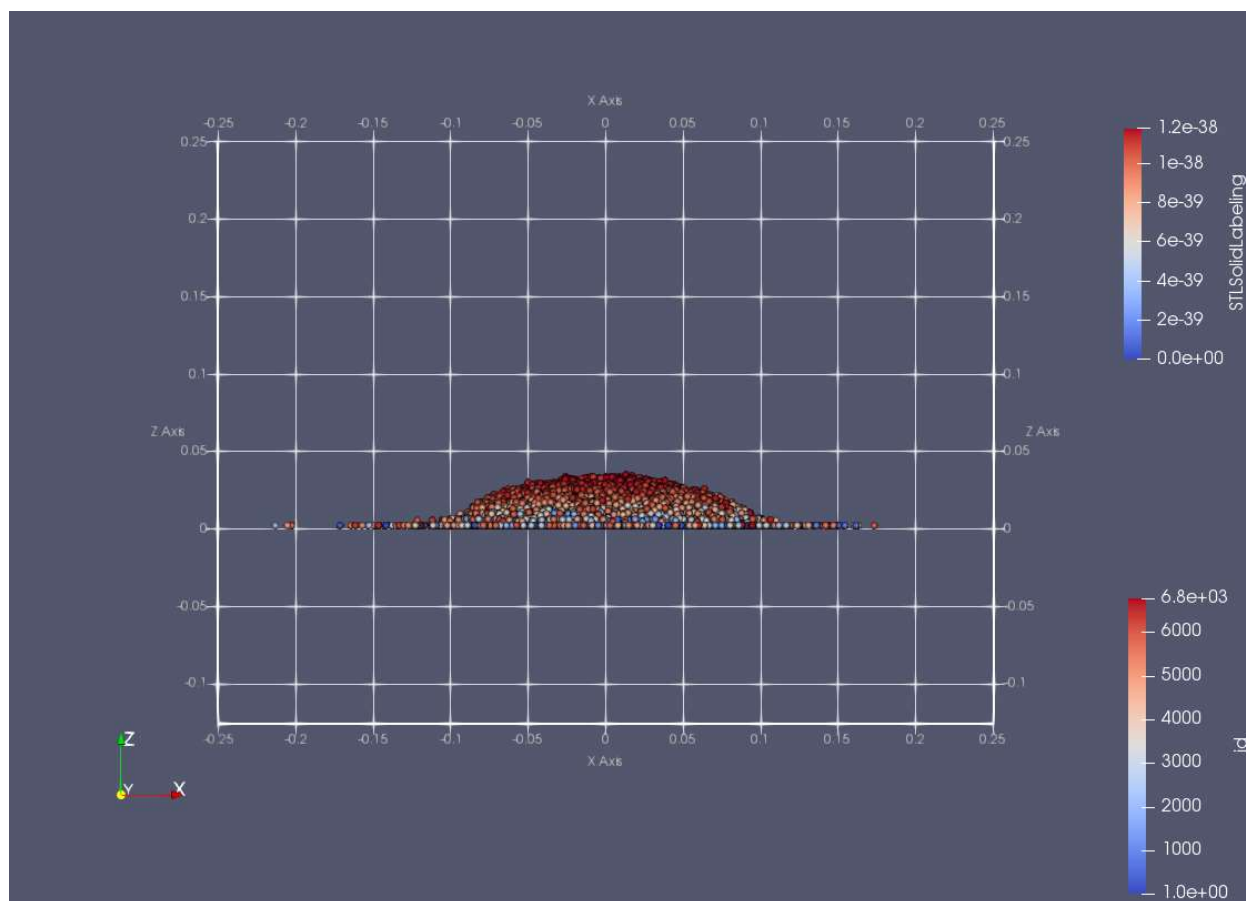


Figure 26: No cohesion, 18.07 deg

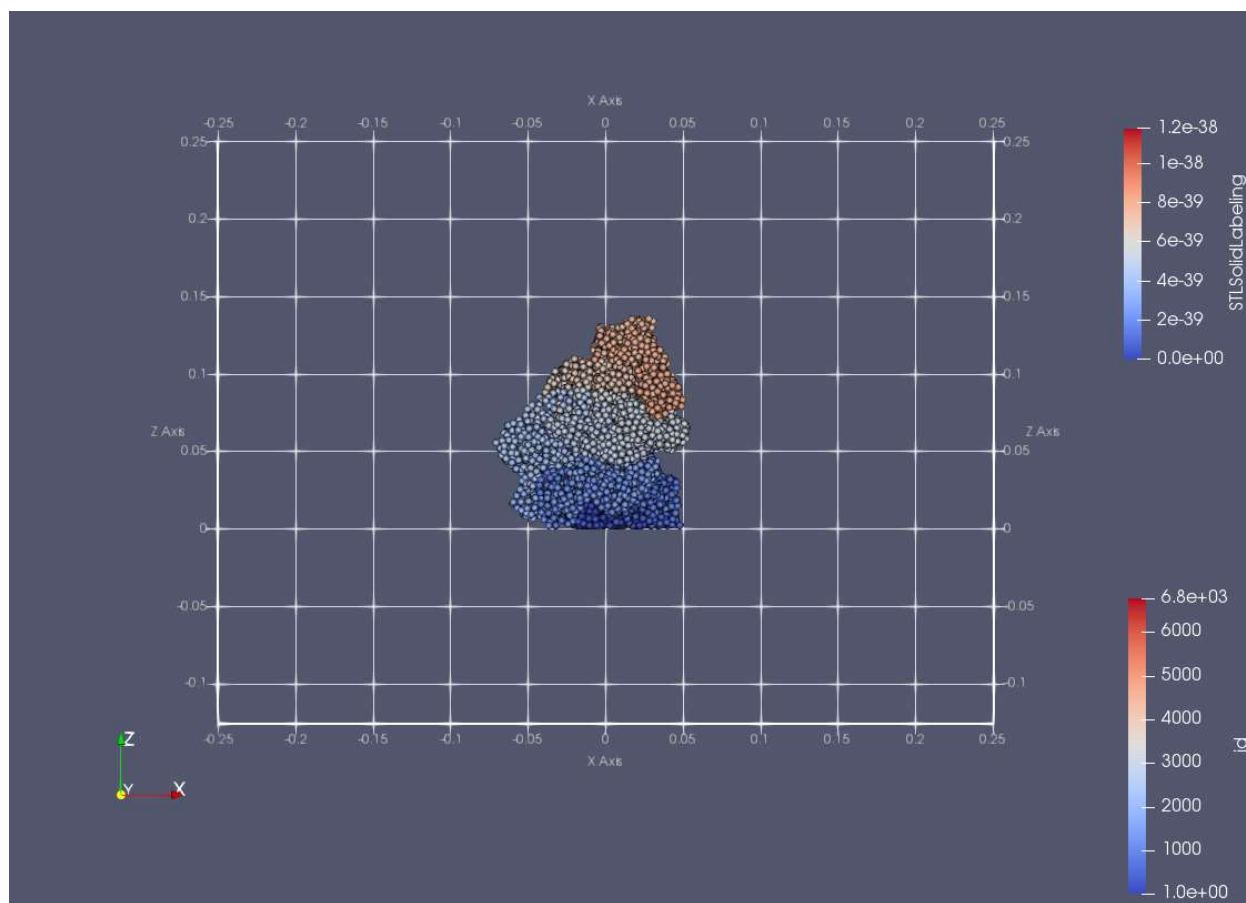


Figure 27: CED = 1000000 ergs/cc, 71.22 deg